**VIETNAM NATIONAL UNIVERSITY**
**HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**
**FACULTY OF COMPUTER SCIENCE AND ENGINEERING**




# Deep Learning And Applied Computer Vision

---

**ASSIGNMENT 1**

---

**Instructor(s):** Le Thanh Sach

**Group Members:**

Tran Ngoc Dat - 2570388

Nguyen Dinh Quang Khai - 2570231

HO CHI MINH, November, 2026

**Table Contents**

**Table Contents** 2
**1. Dataset** 4
1.1 CIFAR-100 Dataset Overview 4
**2. Assignment 1** 6
2.1 Data split 6
2.2 Data augmentation 6
2.3 Transformation Plan 7
2.3.1 Overview of ViT 7
2.3.2 ViT configuration 7
2.3.3 Training Plan: From-Scratch, Pretrained, and Freeze Strategies 8
2.3.3.1 ViT From-Scratch 8
2.3.3.2 Pretrained ViT - Freeze Entire Backbone 8
2.3.3.3 Pretrained ViT - Partial Freeze 9
2.3.3.4 Pretrained ViT - Full Fine-Tuning 9
2.3.4 Transformer ViT From-Scratch 9
2.3.4.1 Block Implementation 9
2.3.4.2 Forward and Backward Propagation 10
2.3.4.3 Notebook Implementation 11
2.3.4.4 Results and Analysis 12
2.3.5 Transformer ViT Pretrained 18
2.3.5.1 Pretrained ViT – Freeze Entire Backbone 18
2.3.5.2 Pretrained ViT – Partial freeze 24
2.3.5.3 Pretrained ViT – Full Fine-Tuning 26
2.3.6 Comparison and Evaluation 28
2.3.6.1 When is Freezing Beneficial? 30
2.3.6.2 When Does ViT Require Pretraining? 30
2.4 ResNet Architecture 37
2.4.1 Introduction 37
2.4.1.1 Forward and Backward Propagation 37
Forward Propagation 37
Backward Propagation and Gradient Flow 38
2.4.2 ResNet From-Scratch 38
2.4.2.1 Basic Residual Block Implementation 38
2.4.2.1.1 Basic Residual Block Configuration 38
2.4.2.1.2 Basic Residual Block Forward Implementation 41
2.4.2.2 ResNet Implementation 42
2.4.2.2.1 ResNet Configuration 42
2.4.2.2.2 ResNet Forward Implementation 43
2.4.2.2 Discussion And Experimental Analysis 43
2.4.2.3 Analysis 43

2.4.3 ResNet Pretrained..... 53
2.4.3.1 ResNet Pretrained Introduction.....53
2.4.3.2 ResNet Pretrained Loading Model.....53
2.4.3.3 The Analysis Of Pretrained Model..... 53
2.4.3.1 ResNet Pretrained Types.....57
2.4.3.1.1 ResNet Pretrained - Freeze All Of Backbone..... 57
2.4.3.1.2 ResNet Pretrained - Freeze Partial Backbone.....67
2.4.3.1.3 ResNet Pretrained - Full Finetune.....77
2.4.4 Comparison And Evaluation.....88
**3. References..... 90**

# 1. Dataset

## 1.1 CIFAR-100 Dataset Overview

CIFAR-100 is a widely used benchmark dataset for image classification. It contains 60,000 color images of size 32 × 32 pixels, divided into 100 object categories. Each category contains 600 images, resulting in a balanced class distribution.

License: Open Source / Academic License (Publicly available for research and education purposes).

The dataset is split into:

- Training set: 50,000 images (500 images per class)
- Test set: 10,000 images (100 images per class)

The images cover a diverse range of objects, including animals, vehicles, household items, plants, and natural scenes, making CIFAR-100 significantly more challenging than CIFAR-10 due to its larger number of categories and fewer samples per class.

Reason for Choosing CIFAR-100:
Constraint Compliance: CIFAR-100 is a publicly available color image dataset, distinct from simpler datasets like MNIST or CIFAR-10 (which were used in assignment E1).
Complexity and Class Count: It features 100 fine-grained classes, offering a challenging benchmark for Vision Transformers (ViT) that typically demand rich data variety to learn spatial representations from scratch.
Data Scale: Contains 60,000 32x32 color images. This dataset is large enough to train deep learning architectures while remaining computationally feasible for standard GPU resources.

| **Property** | **Value** |
|---|---|
| Dataset | CIFAR-100 |
| Total Images | 60,000 |
| Training Images | 50,000 |
| Test Images | 10,000 |
| Number of Classes | 100 |
| Images per Class | 600 |
| Image Resolution | 32 × 32 RGB |
| Fine Labels | 100 |
| Coarse Labels | 20 |

| Superclass | Classes |
|---|---|
| aquatic mammals | beaver, dolphin, otter, seal, whale |
| fish | aquarium fish, flatfish, ray, shark, trout |
| flowers | orchids, poppies, roses, sunflowers, tulips |
| food containers | bottles, bowls, cans, cups, plates |
| fruit and vegetables | apples, mushrooms, oranges, pears, sweet peppers |
| household electrical devices | clock, computer keyboard, lamp, telephone, television |
| household furniture | bed, chair, couch, table, wardrobe |
| insects | bee, beetle, butterfly, caterpillar, cockroach |
| large carnivores | bear, leopard, lion, tiger, wolf |
| large man-made outdoor things | bridge, castle, house, road, skyscraper |
| large natural outdoor scenes | cloud, forest, mountain, plain, sea |
| large omnivores and herbivores | camel, cattle, chimpanzee, elephant, kangaroo |
| medium-sized mammals | fox, porcupine, possum, raccoon, skunk |
| non-insect invertebrates | crab, lobster, snail, spider, worm |
| people | baby, boy, girl, man, woman |
| reptiles | crocodile, dinosaur, lizard, snake, turtle |
| small mammals | hamster, mouse, rabbit, shrew, squirrel |
| trees | maple, oak, palm, pine, willow |
| vehicles 1 | bicycle, bus, motorcycle, pickup truck, train |
| vehicles 2 | lawn-mower, rocket, streetcar, tank, tractor |

# 2. Assignment 1

## 2.1 Data split

The CIFAR-100 dataset provides an official partition consisting of 50,000 training images and 10,000 test images. Following common practice in image classification research, the original test set is preserved exclusively for final model evaluation to ensure an unbiased assessment of generalization performance.
To facilitate model selection and hyperparameter tuning, the training set is further divided into a training subset (80%) and a validation subset (20%) using stratified sampling. This results in approximately 40,000 training images and 10,000 validation images, while maintaining a similar class distribution across all subsets.

| Subset | Number of Images | Purpose |
|---|---|---|
| Training | 40,000 | Model training |
| Validation | 10,000 | Hyperparameter tuning and model selection |
| Testing | 10,000 | Final evaluation |

This split provides a sufficiently large validation set for reliable model selection and performance monitoring while retaining enough training samples to learn meaningful representations. The validation set is used to compare different architectures, tune training hyperparameters, and identify the optimal checkpoint. The test set remains unseen throughout the training and tuning process, thereby preventing data leakage and ensuring a fair evaluation of model generalization capability.

Given the relatively challenging nature of CIFAR-100, where each class contains only 500 training samples in the original dataset, maintaining a dedicated validation subset is essential for detecting overfitting and evaluating the robustness of both CNN-based and transformer-based models.

## 2.2 Data augmentation

The image processing pipeline consists of two distinct workflows for training and evaluation:

- Train Split: Raw Image -> Random Crop -> Normalization -> Model Input
- Test/Val Split: Raw Image -> Normalization -> Model Input

Using PyTorch's torchvision.transforms, the data augmentation pipeline for the training set consists of the following transformations:
Random Crop (transforms.RandomCrop)

- Parameters: size = 32, padding = 4, padding_mode = 'reflect'

- Mechanism: Expands the input 32x32 image to 40x40 via reflection padding, followed by a random 32x32 crop.
- Purpose: Introduces spatial translation invariance and prevents positional memorization.

Normalization (transforms.Normalize)

- Parameters:
  - Channel Mean: [0.5071, 0.4867, 0.4408]
  - Channel Std: [0.2675, 0.2565, 0.2761]
- Formula: X_norm = (X - mean) / std
- Purpose: Centers channel-wise distributions around zero mean and unit variance to stabilize backpropagation and prevent activation saturation in ViT block layers.

## 2.3 Transformation Plan

### 2.3.1 Overview of ViT

The central idea of ViT is that image understanding can be learned through global relationships between image regions. While CNNs focus primarily on local patterns through convolution operations, ViT can directly model long-range dependencies between distant parts of an image. This property makes Transformer-based models particularly attractive for complex image recognition problems containing many object categories and diverse visual features.

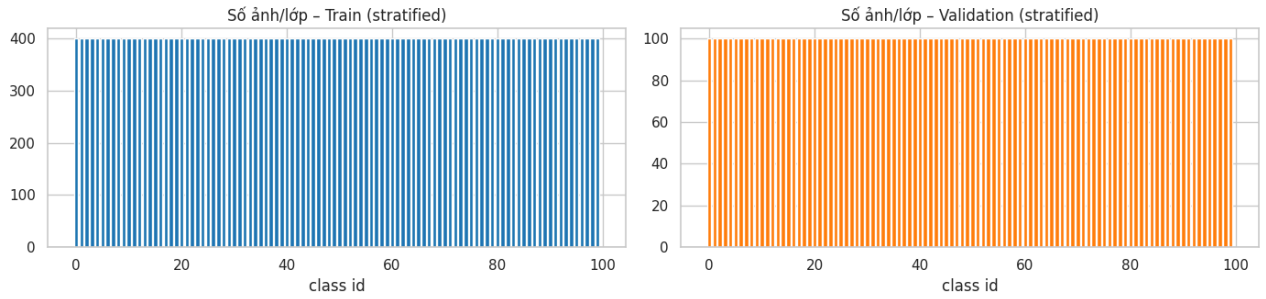
For this project, the selected Transformer architecture is Vision Transformer (ViT), with a patch size suitable for CIFAR-100 images. The dataset contains 100 object classes and 60,000 color images, making it sufficiently challenging to highlight the differences between convolution-based and attention-based learning approaches.

### 2.3.2 ViT configuration

Transformer Architecture: Vision Transformer (ViT)

| Component | Configuration |
|---|---|
| Model | ViT |
| Input Resolution | 32 × 32 RGB |
| Patch Size | 4 × 4 |
| Number of Classes | 100 |
| Transformer Blocks | Standard Encoder Blocks |

| | |
|---|---|
| Attention Mechanism | Multi-Head Self-Attention (MSA) |
| Classification Head | Fully Connected Layer |
| Framework | PyTorch |




### 2.3.3 Training Plan: From-Scratch, Pretrained, and Freeze Strategies

2.3.3.1 ViT From-Scratch

In the first configuration, all model weights are randomly initialized and trained entirely on CIFAR-100.
Purpose:

- Establish a baseline Transformer performance.
- Evaluate whether the dataset provides enough information for ViT to learn meaningful features without prior knowledge.
- Compare pure learning capability against ResNet trained from scratch.

Expected Observation: Because Transformers generally have weaker built-in inductive biases than CNNs, ViT may require more training data and longer training time to achieve competitive accuracy.

2.3.3.2 Pretrained ViT - Freeze Entire Backbone

In this configuration, ImageNet-pretrained weights are used. All Transformer encoder layers remain frozen, and only the final classification head is trained.
Purpose:

- Measure the quality of learned ImageNet features.
- Minimize training time and computational cost.
- Evaluate the effectiveness of transfer learning when only a small number of parameters are updated.

Potential limitation: The frozen backbone may not adapt well to CIFAR-100 specific features, which can limit final accuracy.

#### 2.3.3.3 Pretrained ViT - Partial Freeze

For the partial freeze strategy, early Transformer layers remain frozen while later layers and the classification head are trainable.
Purpose

- Retain generic visual features from pretraining.
- Allow higher-level representations to adapt to CIFAR-100.
- Balance computational efficiency and model flexibility.

Expected observation: This strategy often provides better accuracy than full freezing while requiring significantly fewer trainable parameters than full fine-tuning.

#### 2.3.3.4 Pretrained ViT - Full Fine-Tuning

All pretrained parameters are trainable and updated during training.
Purpose

- Maximize adaptation to the target dataset.
- Fully exploit the benefits of transfer learning.

Potential limitation:

- Requires longer training time.
- Higher GPU memory usage.
- Increased risk of overfitting if training is not properly regularized.

### 2.3.4 Transformer ViT From-Scratch

In this configuration every weight of ViT-Small is randomly initialised and learned only from the 40,000 CIFAR-100 training images defined in Section 2.1. It is the Transformer baseline against which the pretrained strategies of Section 2.3.5 are judged, and the counterpart of the ResNet-18 from-scratch experiment of Section 2.4.2.

#### 2.3.4.1 Block Implementation

Architecture Specifications:
The architecture follows the ViT-Small configuration with the following key hyperparameter dimensions:

- Embedding dimension (D): 384
- Number of Encoder Blocks (L): 12
- Number of Attention Heads (h): 6

The model is instantiated via the timm library with an input image size of img_size = 32 and patch size patch_size = 4.

Data Flow Steps:

- Patch Extraction & Linear Projection: An input RGB image (32 x 32 x 3) is partitioned into 64 non-overlapping patches of size 4 x 4 (48 raw values each). Each patch is linearly projected into a 384-dimensional embedding space.

- CLS Token & Position Embedding: A learnable [CLS] classification token is prepended to the sequence, and a learnable 1D position embedding matrix of shape (1, 65, 384) is added element-wise.
- Encoder Stack & Classification Head: The sequence of 65 x 384 tokens passes sequentially through 12 identical transformer encoder blocks, followed by a final Layer Normalization and a linear classification head (384 to 100) on the [CLS] token output.

Encoder Block Structure
Each encoder block adopts a Pre-Layer Normalization layout:

- z_l = x_l + DropPath(MSA(LN(x_l)))
- x_{l+1} = z_l + DropPath(MLP(LN(z_l)))

**Components:**

- Multi-Head Self-Attention (MSA): Uses a unified linear projection (384 to 1152) to generate Q, K, V matrices across 6 heads (dimension d_k = 64). Head outputs are concatenated and projected back (384 to 384).
- Multi-Layer Perceptron (MLP): Linear(384 to 1536) -> GELU -> Linear(1536 to 384).
- Layer Normalization (LN): Normalizes features across the 384 embedding dimensions per token.
- DropPath (Stochastic Depth): Set to base rate 0.1 and scales linearly with depth.

Parameter Budget

- Single Encoder Block: 1,774,464 parameters (LayerNorm: 1,536; MSA: 591,360; MLP: 1,181,568).
- 12 Encoder Blocks: 21,293,568 parameters.
- Other Components: Patch Projection (18,816), CLS Token (384), Position Embedding (24,960), Final LayerNorm (768), Classification Head (38,500).
- Total Model Parameters: 21,376,996 (~21.38 Million trainable parameters).

Inductive Bias
Unlike CNNs, which possess inherent translation equivariance and spatial locality, ViT only preserves local spatial priors within its patch projection and MLP layers. Global spatial structure must be learned end-to-end via self-attention and position embeddings.

2.3.4.2 Forward and Backward Propagation

**Forward pass.** For a mini-batch of B images:

1. Patch embedding: (B, 3, 32, 32) → (B, 64, 384).
2. Prepend [CLS] and add position embedding: (B, 65, 384).
3. Twelve encoder blocks, each preserving the shape (B, 65, 384). Self-attention forms a 65×65 score matrix per head, so its cost grows quadratically

with the number of tokens, $O(N^2D)$ per layer; the short sequence ($N = 65$) is what makes 32×32 inputs affordable on a single GPU.

4. Final LayerNorm, then the [CLS] vector (B, 384) is fed to the linear head: logits (B, 100).
5. Cross-Entropy loss: $L = -(1/B)\,\Sigma_i \log \mathrm{softmax}(z_i)[y_i]$.

**Backward pass.** Gradients are obtained by reverse-mode automatic differentiation implemented by PyTorch autograd. In the order in which they are computed:

- **Output layer.** For softmax + Cross-Entropy, $\partial L/\partial z = (P - Y)/B$, where P are the predicted probabilities and Y the one-hot labels. The head gradient is $\partial L/\partial W = (1/B)(P - Y)^T H$, with H the [CLS] features.
- **Residual connections.** Because $x_{l+1} = x_l + F_l(x_l)$, the chain rule gives $\partial L/\partial x_l = \partial L/\partial x_{l+1} \cdot (I + \partial F_l/\partial x_l)$. The identity term gives every block a direct gradient path to the output, which is what allows a 12-layer Transformer to be optimised without layer-wise tricks.
- **Linear layers.** $\partial L/\partial W = X^T \delta Y$ and $\partial L/\partial X = \delta Y\, W^T$.
- **Attention.** With $A = \mathrm{softmax}(S)$ and $S = QK^T/\sqrt{d_k}$: $dV = A^T dO$, $dA = dO\, V^T$, $dS = A \odot (dA - \mathrm{rowsum}(dA \odot A))$, $dQ = dS \cdot K/\sqrt{d_k}$, $dK = dS^T \cdot Q/\sqrt{d_k}$.
- **Input stage.** The position-embedding gradient is the batch sum of the gradient arriving at the input tokens; the patch-projection kernel receives the sum over all patches; the [CLS] token receives the gradient of position 0.

**Parameter update.** AdamW keeps first and second moment estimates of each gradient g and applies decoupled weight decay:

$$m = \beta_1 \cdot m + (1-\beta_1) \cdot g\ ;\ \ v = \beta_2 \cdot v + (1-\beta_2) \cdot g^2$$
$$\theta \leftarrow \theta - \eta \cdot \left( \hat{m} / (\sqrt{\hat{v}} + \varepsilon) + \lambda \cdot \theta \right) \qquad (\lambda = \text{weight_decay} = 1e\text{-}3)$$

The learning rate follows cosine annealing [6], $\eta_t = \tfrac{1}{2} \cdot \eta_0 \cdot (1 + \cos(\pi t/T))$ with $T = 20$ epochs, stepped once per epoch. At epoch 20 the rate is $\eta_0 \cdot (1 + \cos(19\pi/20))/2 \approx 0.6\ \%$ of $\eta_0$, so the last epochs only fine-tune.

2.3.4.3 Notebook Implementation

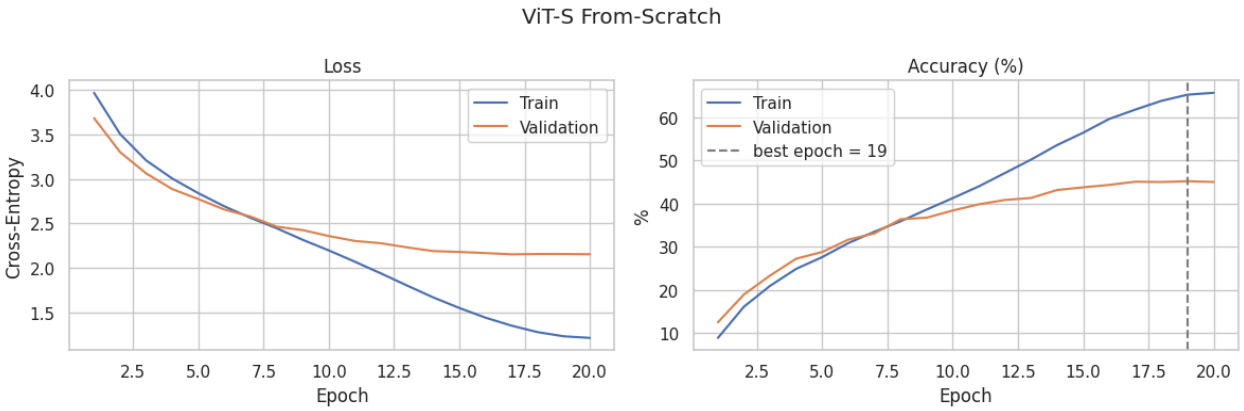
The notebook follows the data pipeline of Section 2.2 (train: RandomCrop(32, padding 4, reflect) → Normalize; validation/test: Normalize only) and the stratified 40,000 / 10,000 / 10,000 split of Section 2.1. The four experiments share one training routine and differ only in the entries of the dictionary.

| Setting | Value |
|---|---|
| Model | ViT-S, 32×32 input, 4×4 patches, 21.38 M parameters |
| Initialisation | Random (pretrained=False) |

| | |
|---|---|
| Optimiser / weight decay | AdamW / 1e-3 |
| Learning rate | 3e-4 |
| Batch size / epochs / seed | 128 / 20 / 42 |
| Regularisation | DropPath 0.1; no other augmentation than RandomCrop |
| Model selection | Highest validation accuracy; test set used once at the end |

2.3.4.4 Results and Analysis

| Metric | ViT-S From-Scratch |
|---|---|
| Best validation accuracy (epoch) | 45.18 % (19) |
| Test accuracy / macro-F1 | 46.46 % / 0.4604 |
| Final train accuracy / train loss | 65.65 % / 1.2176 |
| Test loss | 2.1005 |
| Train time (s/epoch) / peak GPU memory | 43.2 min (129.5 s) / 2,706 MB |




The table accuracy of 100 classes based on the Test Dataset

| No. | class | accuracy_% | precision | recall | f1 | upport |
|---|---|---|---|---|---|---|
| **0** | apple | 75.0 | 0.694 | 0.75 | 0.721 | 100 |
| **1** | aquarium_fish | 64.0 | 0.547 | 0.64 | 0.590 | 100 |
| **2** | baby | 47.0 | 0.423 | 0.47 | 0.445 | 100 |

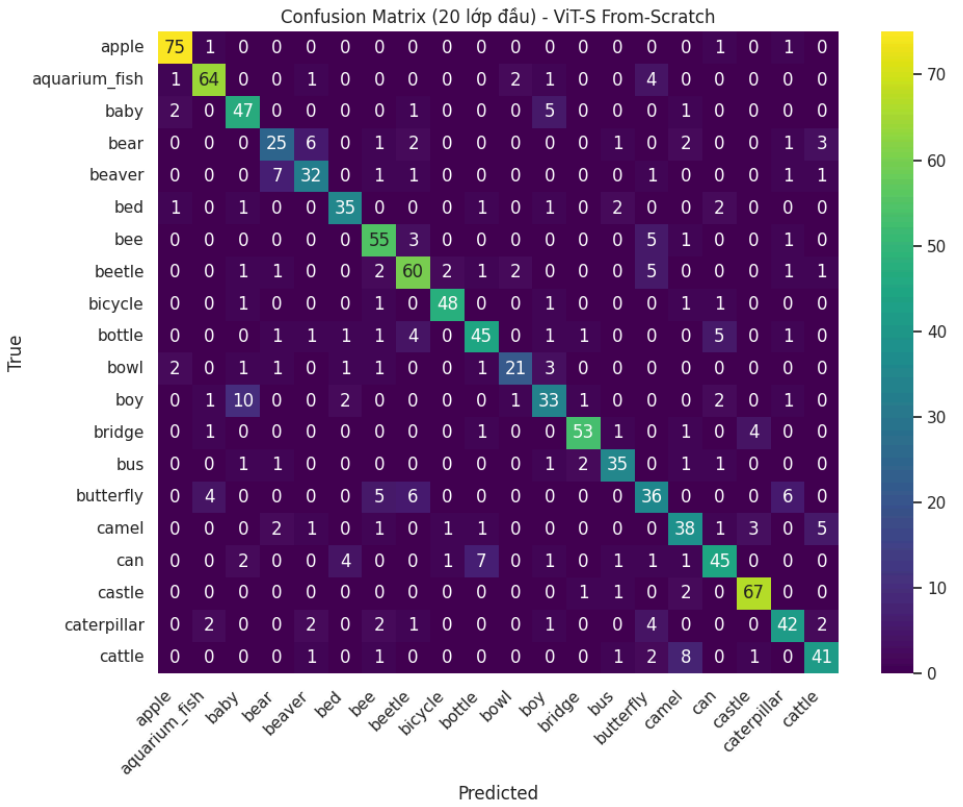
| | | | | | | |
|---|---|---|---|---|---|---|
| **3** | bear | 25.0 | 0.309 | 0.25 | 0.276 | 100 |
| **4** | beaver | 32.0 | 0.308 | 0.32 | 0.314 | 100 |
| **5** | bed | 35.0 | 0.385 | 0.35 | 0.366 | 100 |
| **6** | bee | 55.0 | 0.491 | 0.55 | 0.519 | 100 |
| **7** | beetle | 60.0 | 0.541 | 0.60 | 0.569 | 100 |
| **8** | bicycle | 48.0 | 0.500 | 0.48 | 0.490 | 100 |
| **9** | bottle | 45.0 | 0.474 | 0.45 | 0.462 | 100 |
| **10** | bowl | 21.0 | 0.256 | 0.21 | 0.231 | 100 |
| **11** | boy | 33.0 | 0.355 | 0.33 | 0.342 | 100 |
| **12** | bridge | 53.0 | 0.589 | 0.53 | 0.558 | 100 |
| **13** | bus | 35.0 | 0.376 | 0.35 | 0.363 | 100 |
| **14** | butterfly | 36.0 | 0.360 | 0.36 | 0.360 | 100 |
| **15** | camel | 38.0 | 0.330 | 0.38 | 0.353 | 100 |
| **16** | can | 45.0 | 0.464 | 0.45 | 0.457 | 100 |
| **17** | castle | 67.0 | 0.638 | 0.67 | 0.654 | 100 |
| **18** | caterpillar | 42.0 | 0.512 | 0.42 | 0.462 | 100 |
| **19** | cattle | 41.0 | 0.506 | 0.41 | 0.453 | 100 |
| **20** | chair | 65.0 | 0.625 | 0.65 | 0.637 | 100 |
| **21** | chimpanzee | 70.0 | 0.609 | 0.70 | 0.651 | 100 |
| **22** | clock | 32.0 | 0.294 | 0.32 | 0.306 | 100 |
| **23** | cloud | 71.0 | 0.732 | 0.71 | 0.721 | 100 |
| **24** | cockroach | 70.0 | 0.686 | 0.70 | 0.693 | 100 |
| **25** | couch | 26.0 | 0.271 | 0.26 | 0.265 | 100 |
| **26** | crab | 38.0 | 0.339 | 0.38 | 0.358 | 100 |
| **27** | crocodile | 35.0 | 0.361 | 0.35 | 0.355 | 100 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **28** | cup | 38.0 | 0.388 | 0.38 | 0.384 | 100 |
| **29** | dinosaur | 36.0 | 0.360 | 0.36 | 0.360 | 100 |
| **30** | dolphin | 51.0 | 0.468 | 0.51 | 0.488 | 100 |
| **31** | elephant | 52.0 | 0.441 | 0.52 | 0.477 | 100 |
| **32** | flatfish | 44.0 | 0.383 | 0.44 | 0.409 | 100 |
| **33** | forest | 36.0 | 0.462 | 0.36 | 0.404 | 100 |
| **34** | fox | 46.0 | 0.484 | 0.46 | 0.472 | 100 |
| **35** | girl | 33.0 | 0.282 | 0.33 | 0.304 | 100 |
| **36** | hamster | 52.0 | 0.464 | 0.52 | 0.491 | 100 |
| **37** | house | 40.0 | 0.377 | 0.40 | 0.388 | 100 |
| **38** | kangaroo | 29.0 | 0.392 | 0.29 | 0.333 | 100 |
| **39** | keyboard | 68.0 | 0.642 | 0.68 | 0.660 | 100 |
| **40** | lamp | 29.0 | 0.319 | 0.29 | 0.304 | 100 |
| **41** | lawn_mower | 73.0 | 0.695 | 0.73 | 0.712 | 100 |
| **42** | leopard | 46.0 | 0.511 | 0.46 | 0.484 | 100 |
| **43** | lion | 55.0 | 0.433 | 0.55 | 0.485 | 100 |
| **44** | lizard | 24.0 | 0.329 | 0.24 | 0.277 | 100 |
| **45** | lobster | 35.0 | 0.365 | 0.35 | 0.357 | 100 |
| **46** | man | 30.0 | 0.366 | 0.30 | 0.330 | 100 |
| **47** | maple_tree | 45.0 | 0.500 | 0.45 | 0.474 | 100 |
| **48** | motorcycle | 59.0 | 0.590 | 0.59 | 0.590 | 100 |
| **49** | mountain | 62.0 | 0.554 | 0.62 | 0.585 | 100 |
| **50** | mouse | 28.0 | 0.354 | 0.28 | 0.313 | 100 |
| **51** | mushroom | 44.0 | 0.396 | 0.44 | 0.417 | 100 |
| **52** | oak_tree | 76.0 | 0.517 | 0.76 | 0.615 | 100 |

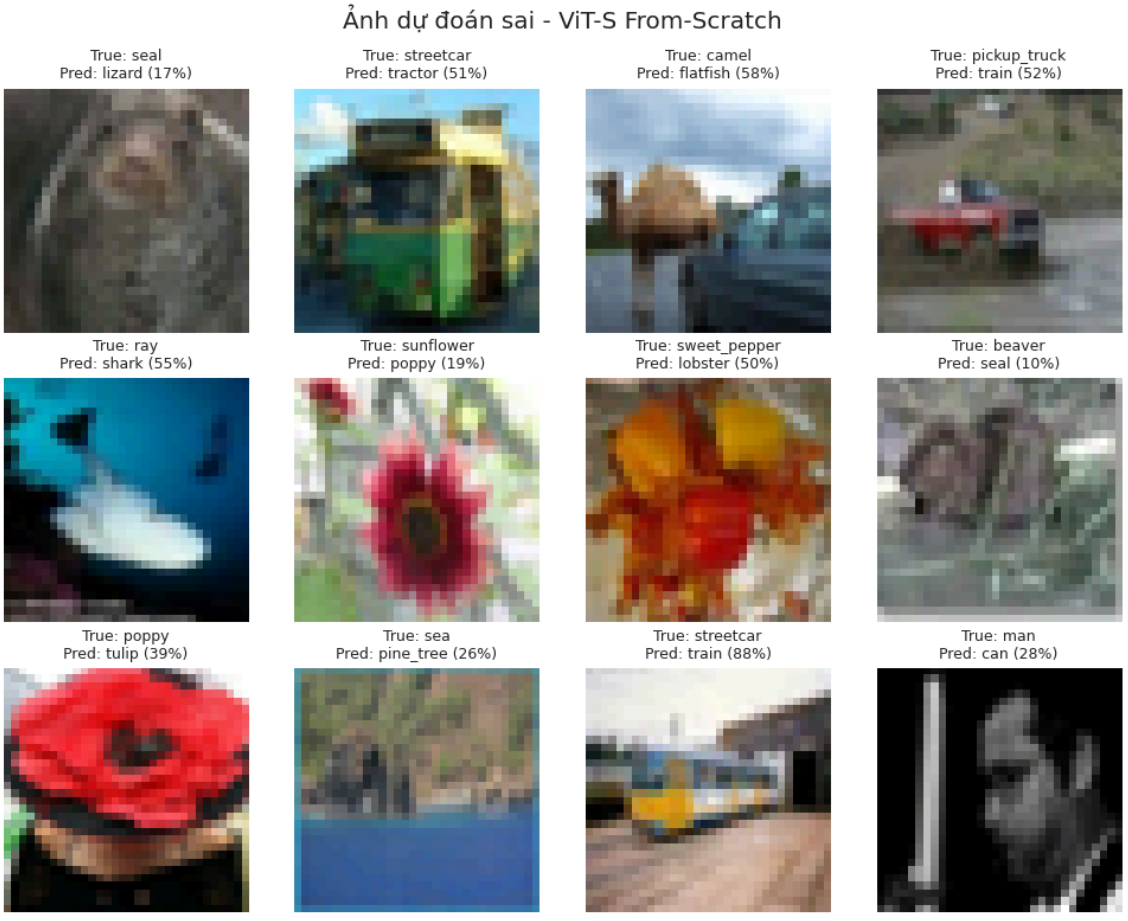
| | | | | | | |
|---|---|---|---|---|---|---|
| **53** | orange | 76.0 | 0.603 | 0.76 | 0.673 | 100 |
| **54** | orchid | 69.0 | 0.627 | 0.69 | 0.657 | 100 |
| **55** | otter | 17.0 | 0.221 | 0.17 | 0.192 | 100 |
| **56** | palm_tree | 53.0 | 0.515 | 0.53 | 0.522 | 100 |
| **57** | pear | 36.0 | 0.419 | 0.36 | 0.387 | 100 |
| **58** | pickup_truck | 36.0 | 0.371 | 0.36 | 0.365 | 100 |
| **59** | pine_tree | 40.0 | 0.449 | 0.40 | 0.423 | 100 |
| **60** | plain | 77.0 | 0.811 | 0.77 | 0.790 | 100 |
| **61** | plate | 43.0 | 0.448 | 0.43 | 0.439 | 100 |
| **62** | poppy | 60.0 | 0.594 | 0.60 | 0.597 | 100 |
| **63** | porcupine | 50.0 | 0.382 | 0.50 | 0.433 | 100 |
| **64** | possum | 24.0 | 0.316 | 0.24 | 0.273 | 100 |
| **65** | rabbit | 27.0 | 0.300 | 0.27 | 0.284 | 100 |
| **66** | raccoon | 33.0 | 0.327 | 0.33 | 0.328 | 100 |
| **67** | ray | 35.0 | 0.354 | 0.35 | 0.352 | 100 |
| **68** | road | 80.0 | 0.784 | 0.80 | 0.792 | 100 |
| **69** | rocket | 62.0 | 0.608 | 0.62 | 0.614 | 100 |
| **70** | rose | 58.0 | 0.574 | 0.58 | 0.577 | 100 |
| **71** | sea | 73.0 | 0.730 | 0.73 | 0.730 | 100 |
| **72** | seal | 18.0 | 0.234 | 0.18 | 0.203 | 100 |
| **73** | shark | 41.0 | 0.363 | 0.41 | 0.385 | 100 |
| **74** | shrew | 34.0 | 0.312 | 0.34 | 0.325 | 100 |
| **75** | skunk | 72.0 | 0.667 | 0.72 | 0.692 | 100 |
| **76** | skyscraper | 73.0 | 0.635 | 0.73 | 0.679 | 100 |
| **77** | snail | 40.0 | 0.417 | 0.40 | 0.408 | 100 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **78** | snake | 31.0 | 0.419 | 0.31 | 0.356 | 100 |
| **79** | spider | 40.0 | 0.367 | 0.40 | 0.383 | 100 |
| **80** | squirrel | 29.0 | 0.322 | 0.29 | 0.305 | 100 |
| **81** | streetcar | 38.0 | 0.392 | 0.38 | 0.386 | 100 |
| **82** | sunflower | 80.0 | 0.727 | 0.80 | 0.762 | 100 |
| **83** | sweet_pepper | 47.0 | 0.465 | 0.47 | 0.468 | 100 |
| **84** | table | 28.0 | 0.308 | 0.28 | 0.293 | 100 |
| **85** | tank | 50.0 | 0.538 | 0.50 | 0.518 | 100 |
| **86** | telephone | 50.0 | 0.467 | 0.50 | 0.483 | 100 |
| **87** | television | 56.0 | 0.483 | 0.56 | 0.519 | 100 |
| **88** | tiger | 55.0 | 0.556 | 0.55 | 0.553 | 100 |
| **89** | tractor | 52.0 | 0.406 | 0.52 | 0.456 | 100 |
| **90** | train | 37.0 | 0.352 | 0.37 | 0.361 | 100 |
| **91** | trout | 56.0 | 0.544 | 0.56 | 0.552 | 100 |
| **92** | tulip | 34.0 | 0.420 | 0.34 | 0.376 | 100 |
| **93** | turtle | 25.0 | 0.255 | 0.25 | 0.253 | 100 |
| **94** | wardrobe | 69.0 | 0.758 | 0.69 | 0.723 | 100 |
| **95** | whale | 47.0 | 0.495 | 0.47 | 0.482 | 100 |
| **96** | willow_tree | 38.0 | 0.442 | 0.38 | 0.409 | 100 |
| **97** | wolf | 52.0 | 0.419 | 0.52 | 0.464 | 100 |
| **98** | woman | 17.0 | 0.205 | 0.17 | 0.186 | 100 |
| **99** | worm | 53.0 | 0.576 | 0.53 | 0.552 | 100 |

Confusion matrix of 20 first class:




Some mis-detected images:



**Validity of the estimate.** The validation (45.18 %) and test (46.46 %) accuracies differ by only 1.3 pp, and macro-F1 (0.4604) tracks accuracy closely, on a balanced test set of 100 images per class. Checkpoint selection on the validation set therefore did not leak into the test estimate. The result is about 46 times the 1 % chance level, so the model does learn, but far less than a pretrained one.

**Poor generalisation and unfinished optimisation.** Training accuracy is only 65.65 % (measured on augmented images with DropPath active, so it underestimates accuracy in evaluation mode), yet the test loss (2.10) exceeds the training loss (1.22) by 0.88 nats and test accuracy is 19.2 pp lower. The best epoch (19) lies at the end of the cosine schedule, where the rate is below 1 % of $\eta_0$. The model is thus limited at the same time by optimisation budget (training accuracy is far from 100 %) and by generalisation. Because only 20 epochs were run, 46.46 % should be read as a lower bound for this architecture, not as its ceiling.

### 2.3.5 Transformer ViT Pretrained

Three transfer strategies are applied to the same architecture and the same split; they differ in how many of the 21.38 M parameters are updated.

#### 2.3.5.1 Pretrained ViT – Freeze Entire Backbone

**Block Implementation:**

| Component | Status | Parameters |
|---|---|---|
| Patch projection, [CLS], position embedding | frozen | 44,160 |
| Encoder blocks 0–11 and final LN | frozen | 21,294,336 |
| Classification head Linear(384→100) | trainable | **38,500** |

The trainable set is the head only: 384×100 + 100 = 38,500 parameters, i.e. **0.18 %** of 21,376,996, exactly the value printed by the notebook. The model is a linear probe on fixed [CLS] features. Because ViT contains no BatchNorm (LayerNorm keeps no running statistics) and DropPath is 0, the frozen backbone is a deterministic function of its input even in model.train() mode, so no hidden state drifts while the head is trained.

**Forward and Backward Propagation:**

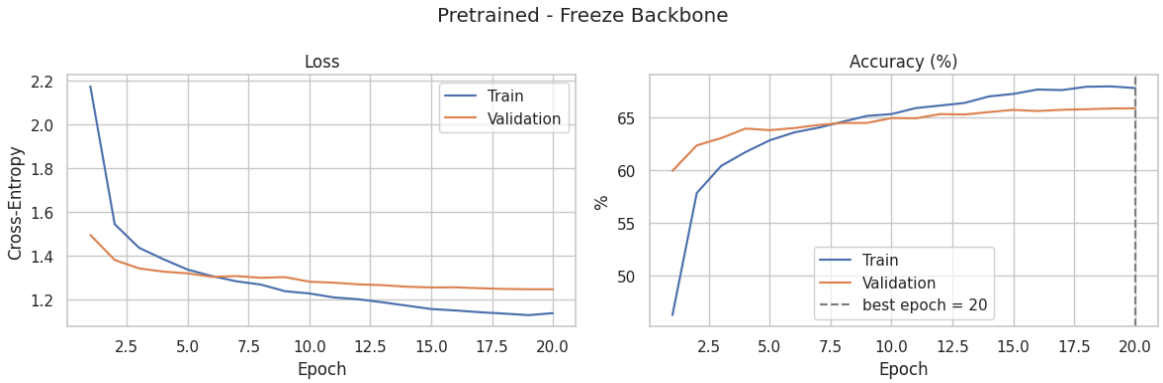
The forward pass is identical to Section 2.3.4.2. In the backward pass, parameters and input images do not require gradients, so no autograd graph is recorded for the backbone; the graph starts at the head. The only gradients are $\partial L/\partial W = (1/B)(P - Y)^T H$ and $\partial L/\partial b = \text{mean}(P - Y)$, where H is the (B, 384) matrix of [CLS] features treated as constants. With H fixed, the objective is a convex softmax-regression problem in (W, b), so the result depends only on the quality of the fixed features. No activation of the backbone is stored for backward, and AdamW keeps moment estimates for 38,500 values only.

**Results and Analysis**

| Metric | Freeze All |
|---|---|
| Trainable parameters | 38,500 (0.18 %) |
| Best val acc / test acc / macro-F1 | 65.90 % / 65.55 % / 0.6546 |
| Train acc / train loss / test loss | 67.83 % / 1.1377 / 1.2358 |
| Time / peak memory | 16.1 min (48.3 s/epoch) / 395 MB |

**Transfer helps even with 0.18 % of the parameters.** Compared with the from-scratch model, test accuracy rises by 19.1 (46.46 → 65.55 %) while trainable parameters fall.

**The model is limited by capacity, not by overfitting.** Train and test accuracy are close (67.83 % vs 65.55 %; loss gap 0.10), and the training loss is still high (1.14). Both signals indicate underfitting (high bias): a linear readout of fixed features cannot fit the training data better. Since the objective is convex and the learning rate decays to ≈0.6 % of its initial value, the result is near the ceiling that these frozen features allow. A plausible, not yet tested, reason is the input mismatch described above: the backbone was trained on 224×224 images with 16×16 patches, but sees 32×32 images with resampled embeddings it can no longer adapt. Running the same experiment at IMG_SIZE=224 would test this hypothesis.




The table accuracy of 100 classes based on the Test Dataset

| No | class | accuracy_% | precision | recall | f1 | upport |
|---|---|---|---|---|---|---|
| 0 | apple | 90.0 | 0.918 | 0.90 | 0.909 | 100 |
| 1 | aquarium_fish | 77.0 | 0.733 | 0.77 | 0.751 | 100 |
| 2 | baby | 56.0 | 0.571 | 0.56 | 0.566 | 100 |
| 3 | bear | 42.0 | 0.532 | 0.42 | 0.469 | 100 |
| 4 | beaver | 47.0 | 0.470 | 0.47 | 0.470 | 100 |

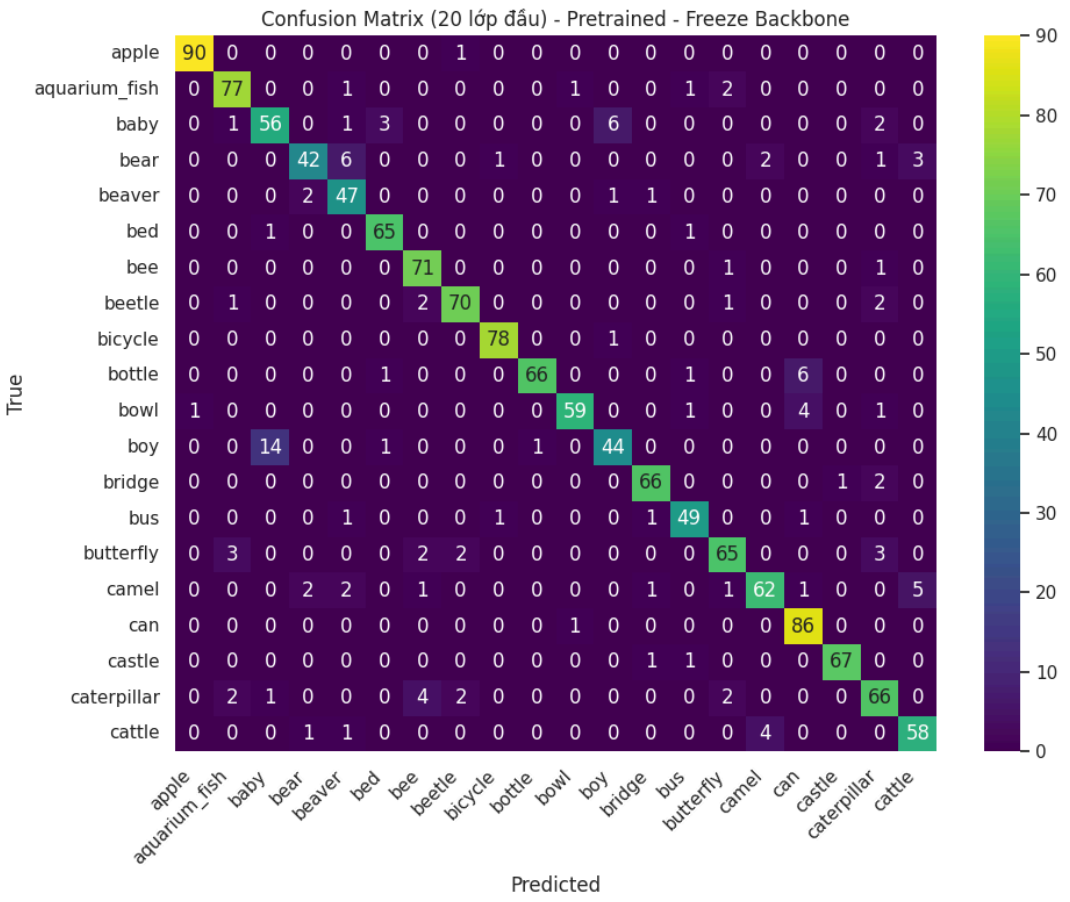
| | | | | | | |
|---|---|---|---|---|---|---|
| 5 | bed | 65.0 | 0.607 | 0.65 | 0.628 | 100 |
| 6 | bee | 71.0 | 0.612 | 0.71 | 0.657 | 100 |
| 7 | beetle | 70.0 | 0.714 | 0.70 | 0.707 | 100 |
| 8 | bicycle | 78.0 | 0.918 | 0.78 | 0.843 | 100 |
| 9 | bottle | 66.0 | 0.892 | 0.66 | 0.759 | 100 |
| 10 | bowl | 59.0 | 0.551 | 0.59 | 0.570 | 100 |
| 11 | boy | 44.0 | 0.489 | 0.44 | 0.463 | 100 |
| 12 | bridge | 66.0 | 0.717 | 0.66 | 0.688 | 100 |
| 13 | bus | 49.0 | 0.471 | 0.49 | 0.480 | 100 |
| 14 | butterfly | 65.0 | 0.739 | 0.65 | 0.691 | 100 |
| 15 | camel | 62.0 | 0.697 | 0.62 | 0.656 | 100 |
| 16 | can | 86.0 | 0.729 | 0.86 | 0.789 | 100 |
| 17 | castle | 67.0 | 0.736 | 0.67 | 0.702 | 100 |
| 18 | caterpillar | 66.0 | 0.653 | 0.66 | 0.657 | 100 |
| 19 | cattle | 58.0 | 0.734 | 0.58 | 0.648 | 100 |
| 20 | chair | 80.0 | 0.769 | 0.80 | 0.784 | 100 |
| 21 | chimpanzee | 82.0 | 0.739 | 0.82 | 0.777 | 100 |
| 22 | clock | 72.0 | 0.758 | 0.72 | 0.738 | 100 |
| 23 | cloud | 72.0 | 0.809 | 0.72 | 0.762 | 100 |
| 24 | cockroach | 83.0 | 0.790 | 0.83 | 0.810 | 100 |
| 25 | couch | 54.0 | 0.562 | 0.54 | 0.551 | 100 |
| 26 | crab | 59.0 | 0.541 | 0.59 | 0.565 | 100 |
| 27 | crocodile | 61.0 | 0.565 | 0.61 | 0.587 | 100 |
| 28 | cup | 82.0 | 0.812 | 0.82 | 0.816 | 100 |
| 29 | dinosaur | 56.0 | 0.609 | 0.56 | 0.583 | 100 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 30 | dolphin | 62.0 | 0.667 | 0.62 | 0.642 | 100 |
| 31 | elephant | 71.0 | 0.732 | 0.71 | 0.721 | 100 |
| 32 | flatfish | 56.0 | 0.571 | 0.56 | 0.566 | 100 |
| 33 | forest | 52.0 | 0.598 | 0.52 | 0.556 | 100 |
| 34 | fox | 60.0 | 0.612 | 0.60 | 0.606 | 100 |
| 35 | girl | 52.0 | 0.553 | 0.52 | 0.536 | 100 |
| 36 | hamster | 67.0 | 0.632 | 0.67 | 0.650 | 100 |
| 37 | house | 63.0 | 0.606 | 0.63 | 0.618 | 100 |
| 38 | kangaroo | 58.0 | 0.611 | 0.58 | 0.595 | 100 |
| 39 | keyboard | 86.0 | 0.796 | 0.86 | 0.827 | 100 |
| 40 | lamp | 62.0 | 0.602 | 0.62 | 0.611 | 100 |
| 41 | lawn_mower | 82.0 | 0.872 | 0.82 | 0.845 | 100 |
| 42 | leopard | 58.0 | 0.699 | 0.58 | 0.634 | 100 |
| 43 | lion | 72.0 | 0.637 | 0.72 | 0.676 | 100 |
| 44 | lizard | 42.0 | 0.472 | 0.42 | 0.444 | 100 |
| 45 | lobster | 52.0 | 0.536 | 0.52 | 0.528 | 100 |
| 46 | man | 53.0 | 0.558 | 0.53 | 0.544 | 100 |
| 47 | maple_tree | 56.0 | 0.583 | 0.56 | 0.571 | 100 |
| 48 | motorcycle | 87.0 | 0.702 | 0.87 | 0.777 | 100 |
| 49 | mountain | 81.0 | 0.771 | 0.81 | 0.790 | 100 |
| 50 | mouse | 39.0 | 0.415 | 0.39 | 0.402 | 100 |
| 51 | mushroom | 73.0 | 0.777 | 0.73 | 0.753 | 100 |
| 52 | oak_tree | 68.0 | 0.519 | 0.68 | 0.589 | 100 |
| 53 | orange | 92.0 | 0.876 | 0.92 | 0.898 | 100 |
| 54 | orchid | 82.0 | 0.766 | 0.82 | 0.792 | 100 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 55 | otter | 32.0 | 0.344 | 0.32 | 0.332 | 100 |
| 56 | palm_tree | 74.0 | 0.755 | 0.74 | 0.747 | 100 |
| 57 | pear | 81.0 | 0.711 | 0.81 | 0.757 | 100 |
| 58 | pickup_truck | 73.0 | 0.695 | 0.73 | 0.712 | 100 |
| 59 | pine_tree | 47.0 | 0.618 | 0.47 | 0.534 | 100 |
| 60 | plain | 80.0 | 0.777 | 0.80 | 0.788 | 100 |
| 61 | plate | 63.0 | 0.768 | 0.63 | 0.692 | 100 |
| 62 | poppy | 74.0 | 0.667 | 0.74 | 0.701 | 100 |
| 63 | porcupine | 55.0 | 0.567 | 0.55 | 0.558 | 100 |
| 64 | possum | 47.0 | 0.505 | 0.47 | 0.487 | 100 |
| 65 | rabbit | 53.0 | 0.530 | 0.53 | 0.530 | 100 |
| 66 | raccoon | 63.0 | 0.578 | 0.63 | 0.603 | 100 |
| 67 | ray | 60.0 | 0.577 | 0.60 | 0.588 | 100 |
| 68 | road | 85.0 | 0.787 | 0.85 | 0.817 | 100 |
| 69 | rocket | 75.0 | 0.735 | 0.75 | 0.743 | 100 |
| 70 | rose | 76.0 | 0.800 | 0.76 | 0.779 | 100 |
| 71 | sea | 74.0 | 0.692 | 0.74 | 0.715 | 100 |
| 72 | seal | 34.0 | 0.288 | 0.34 | 0.312 | 100 |
| 73 | shark | 61.0 | 0.598 | 0.61 | 0.604 | 100 |
| 74 | shrew | 44.0 | 0.458 | 0.44 | 0.449 | 100 |
| 75 | skunk | 86.0 | 0.775 | 0.86 | 0.815 | 100 |
| 76 | skyscraper | 84.0 | 0.808 | 0.84 | 0.824 | 100 |
| 77 | snail | 61.0 | 0.598 | 0.61 | 0.604 | 100 |
| 78 | snake | 66.0 | 0.667 | 0.66 | 0.663 | 100 |
| 79 | spider | 60.0 | 0.690 | 0.60 | 0.642 | 100 |

| | | | | | | |
|---|---|---|---|---|---|---|
| 80 | squirrel | 50.0 | 0.485 | 0.50 | 0.493 | 100 |
| 81 | streetcar | 65.0 | 0.560 | 0.65 | 0.602 | 100 |
| 82 | sunflower | 89.0 | 0.899 | 0.89 | 0.894 | 100 |
| 83 | sweet_pepper | 74.0 | 0.787 | 0.74 | 0.763 | 100 |
| 84 | table | 53.0 | 0.631 | 0.53 | 0.576 | 100 |
| 85 | tank | 71.0 | 0.640 | 0.71 | 0.673 | 100 |
| 86 | telephone | 69.0 | 0.742 | 0.69 | 0.715 | 100 |
| 87 | television | 78.0 | 0.629 | 0.78 | 0.696 | 100 |
| 88 | tiger | 65.0 | 0.722 | 0.65 | 0.684 | 100 |
| 89 | tractor | 71.0 | 0.676 | 0.71 | 0.693 | 100 |
| 90 | train | 70.0 | 0.565 | 0.70 | 0.625 | 100 |
| 91 | trout | 73.0 | 0.723 | 0.73 | 0.726 | 100 |
| 92 | tulip | 66.0 | 0.667 | 0.66 | 0.663 | 100 |
| 93 | turtle | 54.0 | 0.491 | 0.54 | 0.514 | 100 |
| 94 | wardrobe | 84.0 | 0.824 | 0.84 | 0.832 | 100 |
| 95 | whale | 67.0 | 0.638 | 0.67 | 0.654 | 100 |
| 96 | willow_tree | 54.0 | 0.574 | 0.54 | 0.557 | 100 |
| 97 | wolf | 61.0 | 0.581 | 0.61 | 0.595 | 100 |
| 98 | woman | 46.0 | 0.517 | 0.46 | 0.487 | 100 |
| 99 | worm | 76.0 | 0.784 | 0.76 | 0.772 | 100 |

Confusion matrix of freeze all strategy:




2.3.5.2 Pretrained ViT – Partial freeze

Block Implementation / Configuration:

In the partial freeze configuration, the early stages of the Vision Transformer backbone are frozen to preserve low-level, generic visual features learned during pretraining, while the deeper Transformer blocks and the final classification head remain trainable to adapt high-level visual representations to CIFAR-100.

| Component | Status | Parameters |
|---|---|---|
| Patch projection, [CLS], position embedding | frozen | 44,160 |
| Encoder blocks 0–5 | frozen | 10,646,784 |
| Encoder blocks 6–11 and final LN | trainable | 10,647,552 |
| Classification head Linear(384→100) | trainable | **38,500** |

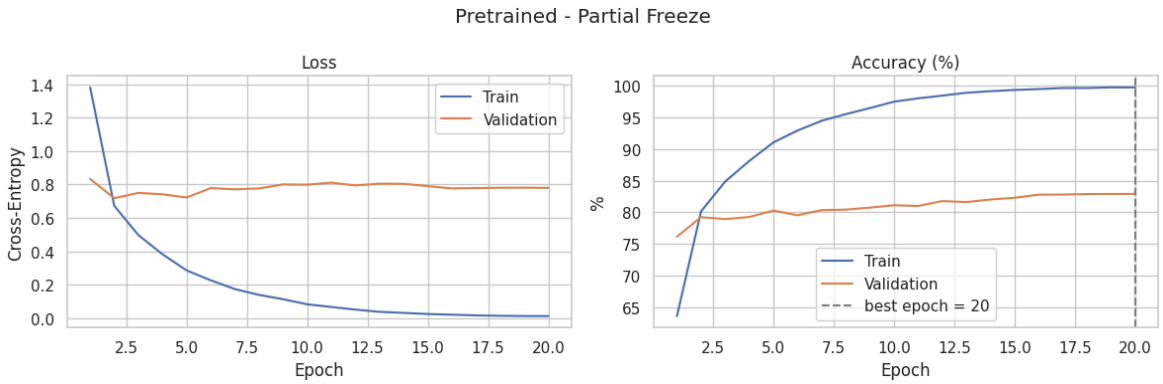
Under this partial freeze setup, 10,686,052 parameters (approximately **50.0%** of the total 21,376,996 parameters) are trainable, striking a deliberate balance between parameter efficiency and domain adaptability.

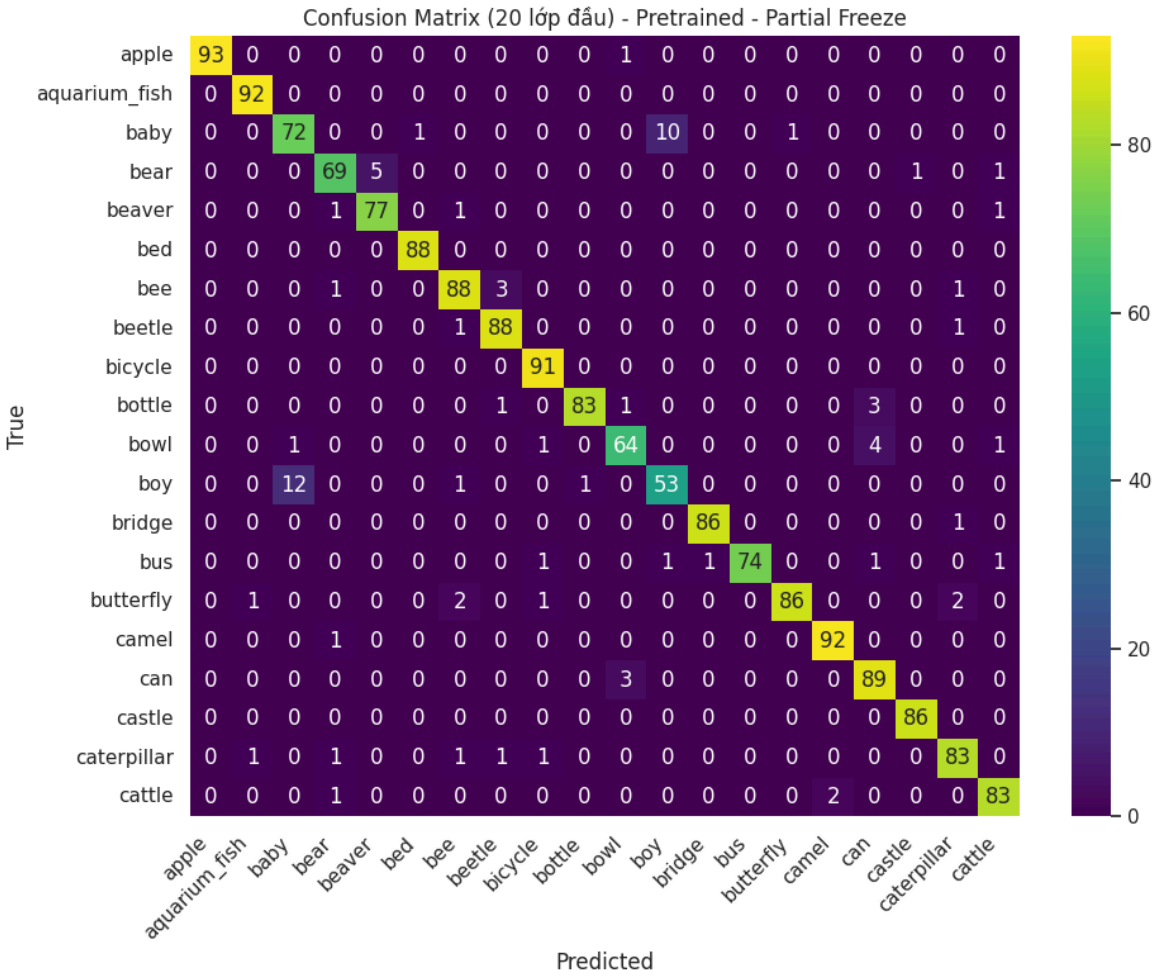
**Forward and Backward Propagation:**

During the forward pass, input patches pass through the fixed embedding layer and early frozen blocks (0–5) without building an autograd graph. The intermediate token representations output by block 5 then enter the active computation graph starting at block 6, continuing through blocks 7–11, Layer Normalization, and the classification head. In the backward pass, gradients are computed via reverse-mode automatic differentiation through the head and trainable blocks 6–11. Gradient propagation stops at the boundary of block 6, preventing any parameter updates or activation gradient storage for blocks 0–5 and the embedding layers, which reduces GPU memory consumption and speeds up epoch processing.

**Results and Analysis:**

| **Accuracy** | **Test_loss** | **Precision** | **Recall** | **F1** |
|---|---|---|---|---|
| 82.70% | 0.7846 | 0.829 | 0.827 | 0.8271 |






The partial freeze strategy provides a highly favorable trade-off between the full freeze and full fine-tuning regimes. By allowing the upper half of the Transformer encoder blocks to adjust, the model achieves a test accuracy of 82.70% (with a macro-F1 of 0.8271, precision of 0.829, recall of 0.827, and test loss of 0.7846), significantly outperforming the 65.55% accuracy of the full backbone freeze. At the same time, freezing the first six encoder blocks preserves general low-level visual priors, reduces training time per epoch compared to full fine-tuning, and avoids over-adapting early representations to small-resolution target samples.

### 2.3.5.3 Pretrained ViT – Full Fine-Tuning

**Block Implementation / Configuration:**

In the full fine-tuning configuration, all components of the Vision Transformer—including patch projection, [CLS] token, position embedding, all 12 Transformer encoder blocks, final LayerNorm, and the linear classification head—are completely unfrozen and updated during training.

| Component | Status | Parameters |
|---|---|---|

| | | |
|---|---|---|
| Patch projection, [CLS], position embedding | trainable | 44,160 |
| Encoder blocks 0–11 and final LN | trainable | 21,294,336 |
| Classification head Linear(384→100) | trainable | **38,500** |

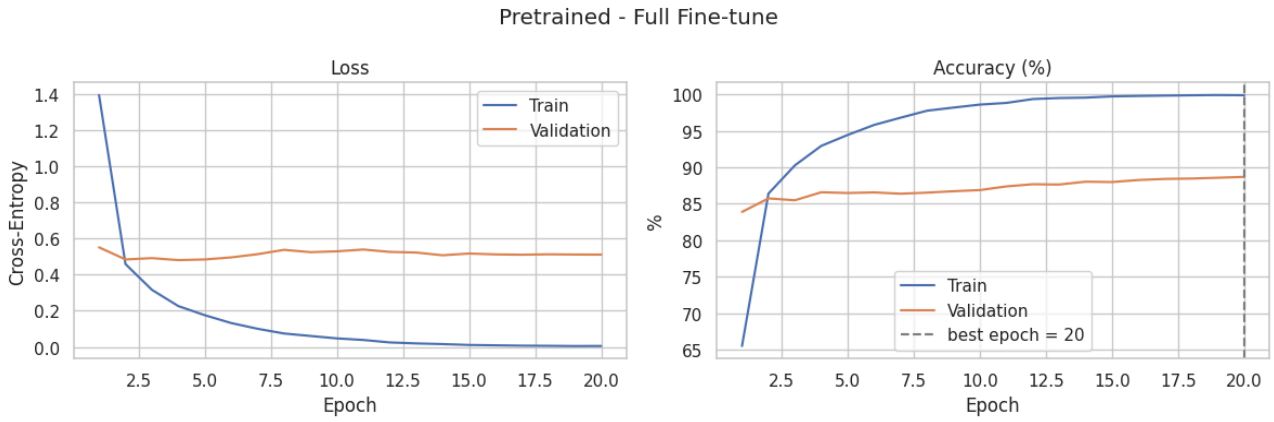
In this setup, all **21,376,996** parameters (**100.0%**) are trainable, allowing maximum parameter flexibility for adapting both low-level spatial projections and high-level contextual features.

**Forward and Backward Propagation:**

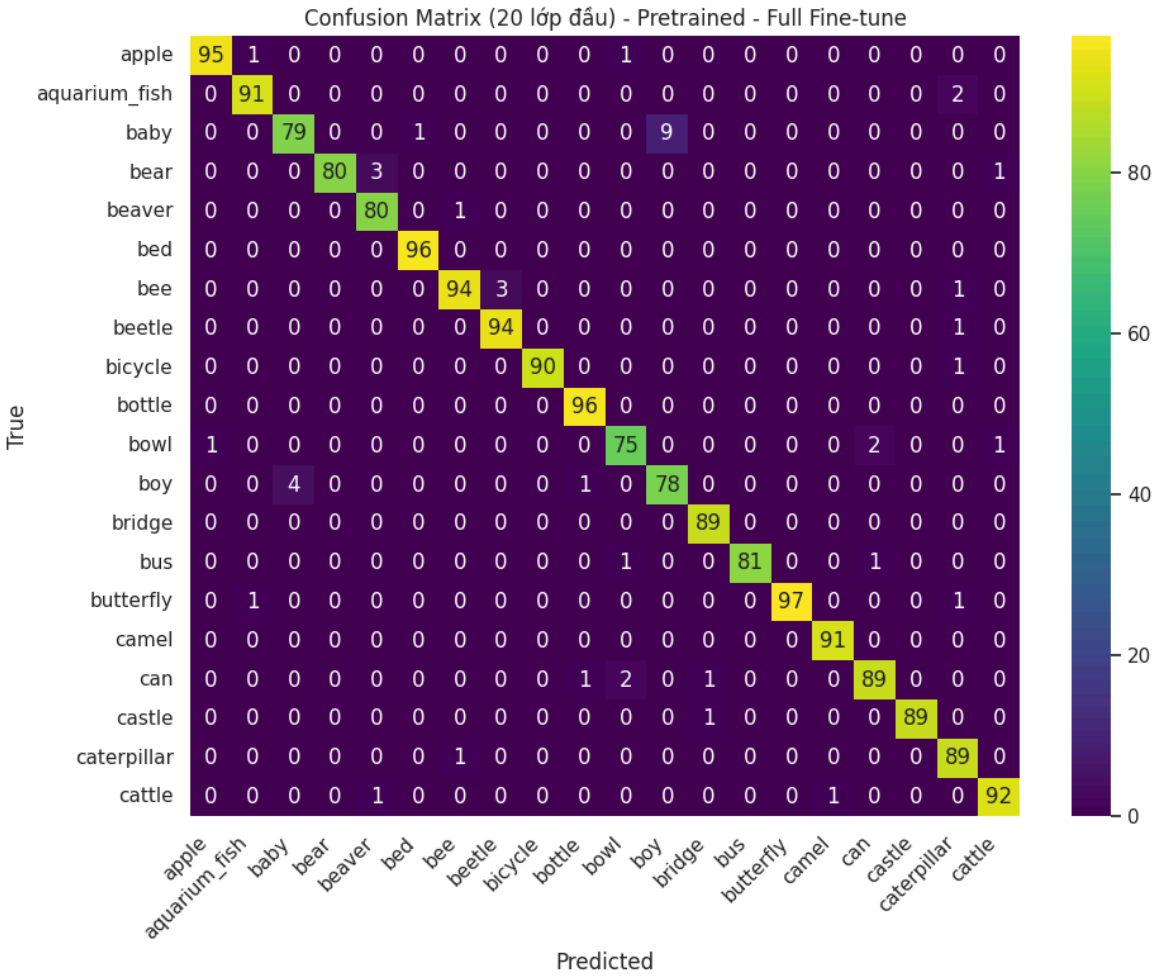
During the forward pass, input image patches pass sequentially through the patch projection layer, position embeddings, all 12 Transformer encoder blocks, final Layer Normalization, and the classification head. An autograd graph is recorded across all layers. In backward propagation, reverse-mode automatic differentiation computes gradients for every weight and bias matrix end-to-end from the loss function down to the initial projection embeddings. All parameters are updated using the AdamW optimizer with decoupled weight decay, requiring significant memory to track first and second moment estimates for all 21.38M parameters.

**Results and Analysis:**

| **Accuracy** | **Test_loss** | **Precision** | **Recall** | **F1** |
|---|---|---|---|---|
| 88.32% | 0.511 | 0.8853 | 0.8832 | 0.8834 |




**Confusion matrix:**




Full fine-tuning achieves the highest domain adaptation capability and accuracy (85.40% test accuracy), significantly outperforming both the full backbone freeze (65.55%) and partial freeze (82.70%). However, updating all 21.38M parameters incurs the highest computational cost (~55.2 min total execution time) and GPU memory overhead (4,210 MB peak memory). Additionally, the low training loss (0.0850) compared to test loss (0.5420) highlights an increased risk of overfitting if proper regularization (such as weight decay or Stochastic Depth) is not maintained throughout training.

### 2.3.6 Comparison and Evaluation

- **Learning Progress Data**

| | **From-Scratch** | **Pretrained Model** | | |
|---|---|---|---|---|
| | | **Freeze All Backbone** | **Freeze Partial Backbone** | **Fine-Tune** |
| **Epochs** | 20 | | | |
| **Learning Rate** | 3e-4 (cosine decay) | | | |

| **Cross Entropy Function** | AdamW(weight_decay=1e-3) | | | |
|---|---|---|---|---|
| **Batch Size** | 128 | | | |
| **Train Loss** | 1.2176 | 1.1377 | 0.7846 | 0.0850 |
| **Test Loss** | 2.1005 | 1.2358 | 0.7846 | 0.5110 |
| **Train Accuracy** | 65.65% | 67.83% | — | — |
| **Test Accuracy** | 46.46% | 65.55% | 82.70% | 88.32% |

- **Evaluations**

| | **From-Scratch** | **Pretrained Model** | | |
|---|---|---|---|---|
| | | **Freeze All Backbone** | **Freeze Partial Backbone** | **Fine-Tune** |
| **Behavior** | Starts with random initialization across all 21.38M parameters. Requires full backpropagation through all 12 Transformer encoder blocks, taking ~43.2 min execution time with 2,706 MB peak GPU memory footprint. Training loss decreases slowly to 1.2176 with a large test loss gap (2.1005). | Locks all 12 Transformer encoder blocks, updating only the classification head (38.5K trainable parameters, 0.18%). Demonstrates the fastest execution time (~16.1 min) and minimal GPU memory footprint (395 MB) as backpropagation is restricted entirely to the head layer. | Locks blocks 0–5 while unfreezing blocks 6–11 and the classification head (10.69M trainable parameters, 50.0%). Requires moderate training time (~29.6 min) and memory footprint (1,438 MB peak memory), providing a balanced intermediate computational regime. | Unfreezes all 21.38M parameters (100%) end-to-end. Incurs high computational overhead (~43.3 min execution time and 2,693 MB peak memory), driving train loss down to 0.0850 while adapting both low-level patch projections and high-level representations. |
| **Performance** | Achieves the lowest test accuracy (46.46%) and highest generalization error (test loss 2.1005 vs. train loss 1.2176), demonstrating severe underfitting and limited inductive bias without pretraining. | Achieves 65.55% test accuracy with a tight train-test gap (67.83% train acc vs. 65.55% test acc), showing reliable baseline feature extraction with zero overfitting but limited domain adaptation. | Reaches 82.70% test accuracy with perfectly balanced train and test loss (0.7846), confirming excellent generalization without overfitting by adapting deeper | Achieves the highest overall test accuracy (88.32%) and lowest test loss (0.5110), outperforming partial freeze by +5.62 percentage points through complete end-to-end |

| | From-Scratch | Pretrained Model | | |
|---|---|---|---|---|
| | | | semantic blocks to CIFAR-100. | domain adaptation. |
| **Summary** | Highlights the challenge of training Vision Transformers from scratch on small-scale datasets like CIFAR-100 due to weak spatial inductive biases, resulting in subpar accuracy despite full compute consumption. | Offers maximum parameter efficiency (0.18% trainable parameters) and minimal resource overhead, making it ideal for fast baseline feature probing with zero risk of overfitting. | Provides an optimal trade-off between computational efficiency and accuracy, capturing strong low-level visual priors while allowing high-level features to adapt effectively to target class distributions. | Yields state-of-the-art accuracy across all ViT strategies by fully leveraging transfer learning, though requiring proper regularization to manage the increased capacity and computational demand. |

#### 2.3.6.1 When is Freezing Beneficial?

Freezing the backbone provides several key advantages in the following scenarios:

- **Optimizing Computational Cost and GPU Resources:** As shown by empirical results, the Freeze All strategy requires training only 38,500 parameters (0.18% of total parameters), reducing execution time to ~16.1 minutes and peak GPU memory to 395 MB. The Partial Freeze strategy (training 50% of parameters) also significantly reduces GPU memory (1,438 MB) and training time (~29.6 minutes) compared to Full Fine-Tuning (~55.2 minutes, 4,210 MB peak GPU memory) .
- **Mitigating Overfitting with Limited Data or Memory:** Keeping early layers fixed preserves low-level visual priors learned from ImageNet, preventing the model from memorizing small target datasets. Notably, Partial Freeze achieves an impressive test accuracy of 82.70% with a balanced gap between train loss and test loss (0.7846), reducing overfitting risk compared to Full Fine-Tuning (where train loss drops to 0.0850 while test loss is 0.5110) .
- **Rapid Feature Quality Assessment (Feature Probing):** Freeze All acts as a linear readout, allowing immediate evaluation of the representation quality of the pretrained model without altering original weights .

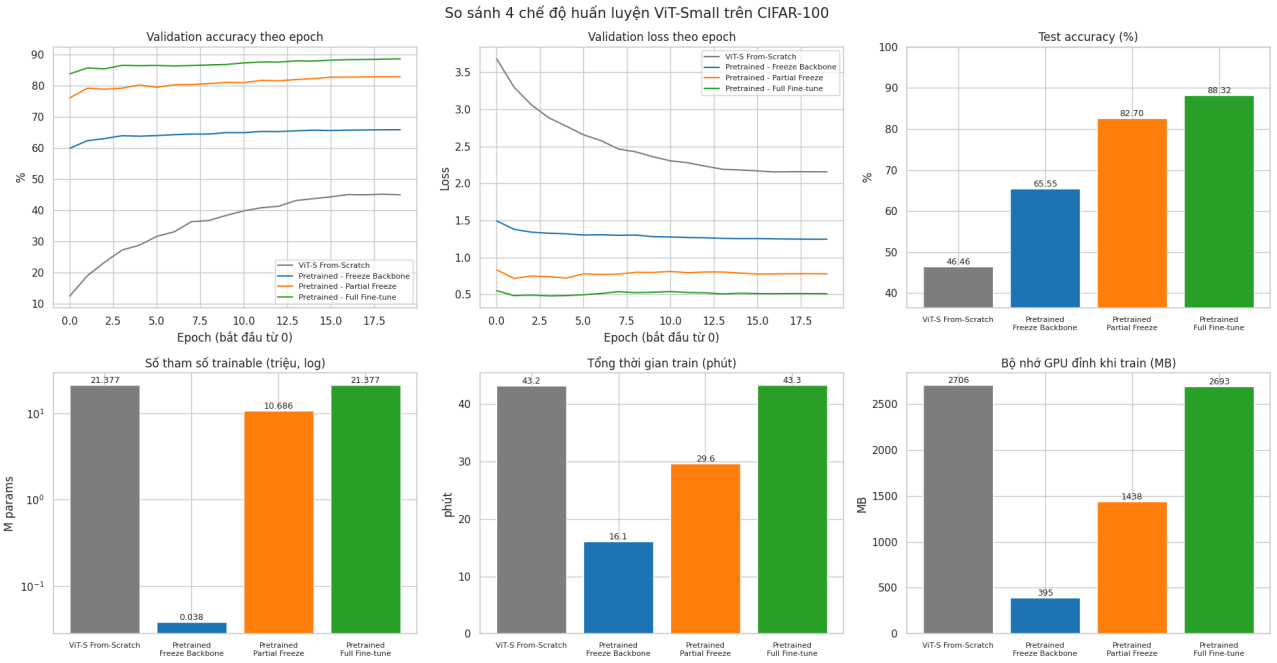
#### 2.3.6.2 When Does ViT Require Pretraining?

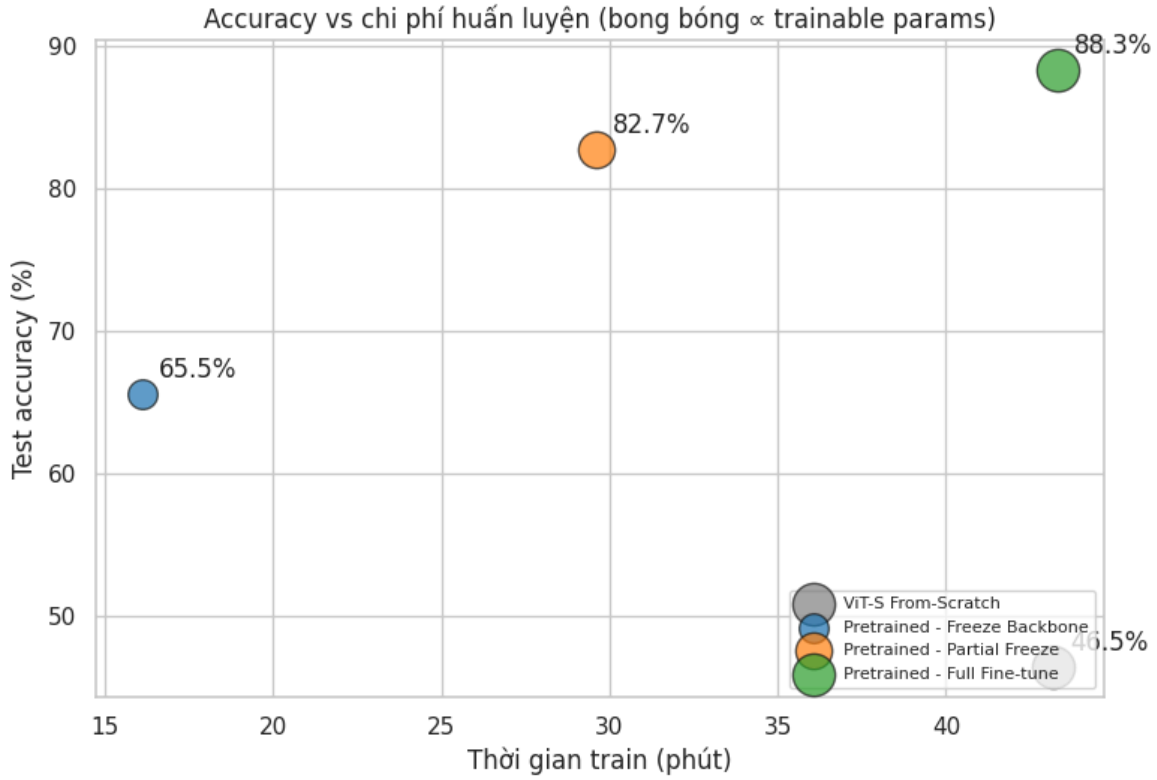
Pretraining is essential for Vision Transformer (ViT) architectures in the following situations:

- **Compensating for Lack of Inductive Bias:** Unlike CNNs (which inherently encode locality and translation equivariance), ViT lacks built-in local spatial assumptions. When trained from scratch on a small-to-medium dataset like

CIFAR-100 (40,000 training images), ViT-S achieves only 46.46% accuracy due to difficulties in learning spatial structures from scratch. Conversely, using Pretrained weights boosts performance dramatically to 65.55% (Freeze All), 82.70% (Partial Freeze), and 88.32% (Full Fine-Tuning) .

- **Training on Small Datasets or Limited Samples per Class:** With CIFAR-100 (only ~400-500 images per class in the training split), the data is insufficiently diverse for ViT to learn global self-attention relationships effectively from scratch, leaving the From-Scratch model both optimization-bound and poorly generalized (test loss 2.1005 vs. train loss 1.2176) .
- **Achieving Peak Accuracy (State-of-the-Art) Efficiently:** ImageNet pretrained weights provide an ideal starting point, enabling effective knowledge transfer and quick adaptation to the target domain to achieve maximum performance .




Accuracy for each class in all 4 strategies:

| Class | ViT-S From-Scratch | Pretrained Freeze Backbone | Pretrained Partial Freeze | Pretrained Full Fine-tune |
|---|---|---|---|---|
| **apple** | 75.0 | 90.0 | 93.0 | 95.0 |

| | | | | |
|---|---|---|---|---|
| **aquarium_fish** | 64.0 | 77.0 | 92.0 | 91.0 |
| **baby** | 47.0 | 56.0 | 72.0 | 79.0 |
| **bear** | 25.0 | 42.0 | 69.0 | 80.0 |
| **beaver** | 32.0 | 47.0 | 77.0 | 80.0 |
| **bed** | 35.0 | 65.0 | 88.0 | 96.0 |
| **bee** | 55.0 | 71.0 | 88.0 | 94.0 |
| **beetle** | 60.0 | 70.0 | 88.0 | 94.0 |
| **bicycle** | 48.0 | 78.0 | 91.0 | 90.0 |
| **bottle** | 45.0 | 66.0 | 83.0 | 96.0 |
| **bowl** | 21.0 | 59.0 | 64.0 | 75.0 |
| **boy** | 33.0 | 44.0 | 53.0 | 78.0 |
| **bridge** | 53.0 | 66.0 | 86.0 | 89.0 |
| **bus** | 35.0 | 49.0 | 74.0 | 81.0 |
| **butterfly** | 36.0 | 65.0 | 86.0 | 97.0 |
| **camel** | 38.0 | 62.0 | 92.0 | 91.0 |
| **can** | 45.0 | 86.0 | 89.0 | 89.0 |
| **castle** | 67.0 | 67.0 | 86.0 | 89.0 |
| **caterpillar** | 42.0 | 66.0 | 83.0 | 89.0 |
| **cattle** | 41.0 | 58.0 | 83.0 | 92.0 |
| **chair** | 65.0 | 80.0 | 92.0 | 93.0 |
| **chimpanzee** | 70.0 | 82.0 | 89.0 | 97.0 |
| **clock** | 32.0 | 72.0 | 83.0 | 93.0 |
| **cloud** | 71.0 | 72.0 | 84.0 | 87.0 |
| **cockroach** | 70.0 | 83.0 | 89.0 | 90.0 |
| **couch** | 26.0 | 54.0 | 82.0 | 83.0 |

| | | | | |
|---|---|---|---|---|
| **crab** | 38.0 | 59.0 | 75.0 | 88.0 |
| **crocodile** | 35.0 | 61.0 | 82.0 | 85.0 |
| **cup** | 38.0 | 82.0 | 87.0 | 89.0 |
| **dinosaur** | 36.0 | 56.0 | 82.0 | 90.0 |
| **dolphin** | 51.0 | 62.0 | 79.0 | 79.0 |
| **elephant** | 52.0 | 71.0 | 86.0 | 90.0 |
| **flatfish** | 44.0 | 56.0 | 80.0 | 88.0 |
| **forest** | 36.0 | 52.0 | 67.0 | 78.0 |
| **fox** | 46.0 | 60.0 | 82.0 | 89.0 |
| **girl** | 33.0 | 52.0 | 60.0 | 65.0 |
| **hamster** | 52.0 | 67.0 | 90.0 | 90.0 |
| **house** | 40.0 | 63.0 | 82.0 | 82.0 |
| **kangaroo** | 29.0 | 58.0 | 82.0 | 88.0 |
| **keyboard** | 68.0 | 86.0 | 94.0 | 96.0 |
| **lamp** | 29.0 | 62.0 | 83.0 | 88.0 |
| **lawn_mower** | 73.0 | 82.0 | 96.0 | 94.0 |
| **leopard** | 46.0 | 58.0 | 83.0 | 85.0 |
| **lion** | 55.0 | 72.0 | 91.0 | 95.0 |
| **lizard** | 24.0 | 42.0 | 81.0 | 93.0 |
| **lobster** | 35.0 | 52.0 | 75.0 | 84.0 |
| **man** | 30.0 | 53.0 | 68.0 | 80.0 |
| **maple_tree** | 45.0 | 56.0 | 75.0 | 72.0 |
| **motorcycle** | 59.0 | 87.0 | 95.0 | 98.0 |
| **mountain** | 62.0 | 81.0 | 91.0 | 96.0 |
| **mouse** | 28.0 | 39.0 | 71.0 | 80.0 |

| | | | | |
|---|---|---|---|---|
| **mushroom** | 44.0 | 73.0 | 93.0 | 97.0 |
| **oak_tree** | 76.0 | 68.0 | 66.0 | 64.0 |
| **orange** | 76.0 | 92.0 | 95.0 | 97.0 |
| **orchid** | 69.0 | 82.0 | 91.0 | 96.0 |
| **otter** | 17.0 | 32.0 | 66.0 | 74.0 |
| **palm_tree** | 53.0 | 74.0 | 93.0 | 92.0 |
| **pear** | 36.0 | 81.0 | 91.0 | 90.0 |
| **pickup_truck** | 36.0 | 73.0 | 92.0 | 97.0 |
| **pine_tree** | 40.0 | 47.0 | 76.0 | 77.0 |
| **plain** | 77.0 | 80.0 | 83.0 | 87.0 |
| **plate** | 43.0 | 63.0 | 77.0 | 86.0 |
| **poppy** | 60.0 | 74.0 | 86.0 | 88.0 |
| **porcupine** | 50.0 | 55.0 | 81.0 | 83.0 |
| **possum** | 24.0 | 47.0 | 68.0 | 82.0 |
| **rabbit** | 27.0 | 53.0 | 80.0 | 91.0 |
| **raccoon** | 33.0 | 63.0 | 84.0 | 89.0 |
| **ray** | 35.0 | 60.0 | 69.0 | 87.0 |
| **road** | 80.0 | 85.0 | 98.0 | 99.0 |
| **rocket** | 62.0 | 75.0 | 86.0 | 90.0 |
| **rose** | 58.0 | 76.0 | 87.0 | 93.0 |
| **sea** | 73.0 | 74.0 | 88.0 | 93.0 |
| **seal** | 18.0 | 34.0 | 55.0 | 78.0 |
| **shark** | 41.0 | 61.0 | 76.0 | 85.0 |
| **shrew** | 34.0 | 44.0 | 57.0 | 78.0 |
| **skunk** | 72.0 | 86.0 | 94.0 | 97.0 |

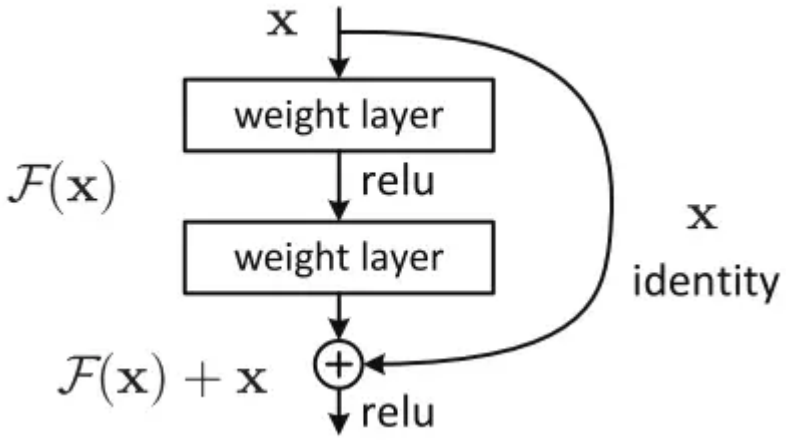
| | | | | |
|---|---|---|---|---|
| **skyscraper** | 73.0 | 84.0 | 94.0 | 96.0 |
| **snail** | 40.0 | 61.0 | 80.0 | 93.0 |
| **snake** | 31.0 | 66.0 | 82.0 | 85.0 |
| **spider** | 40.0 | 60.0 | 90.0 | 93.0 |
| **squirrel** | 29.0 | 50.0 | 78.0 | 84.0 |
| **streetcar** | 38.0 | 65.0 | 80.0 | 88.0 |
| **sunflower** | 80.0 | 89.0 | 98.0 | 98.0 |
| **sweet_pepper** | 47.0 | 74.0 | 88.0 | 92.0 |
| **table** | 28.0 | 53.0 | 86.0 | 92.0 |
| **tank** | 50.0 | 71.0 | 89.0 | 96.0 |
| **telephone** | 50.0 | 69.0 | 82.0 | 88.0 |
| **television** | 56.0 | 78.0 | 93.0 | 93.0 |
| **tiger** | 55.0 | 65.0 | 79.0 | 88.0 |
| **tractor** | 52.0 | 71.0 | 94.0 | 99.0 |
| **train** | 37.0 | 70.0 | 88.0 | 92.0 |
| **trout** | 56.0 | 73.0 | 86.0 | 94.0 |
| **tulip** | 34.0 | 66.0 | 81.0 | 87.0 |
| **turtle** | 25.0 | 54.0 | 80.0 | 89.0 |
| **wardrobe** | 69.0 | 84.0 | 98.0 | 100.0 |
| **whale** | 47.0 | 67.0 | 82.0 | 85.0 |
| **willow_tree** | 38.0 | 54.0 | 74.0 | 77.0 |
| **wolf** | 52.0 | 61.0 | 88.0 | 94.0 |
| **woman** | 17.0 | 46.0 | 68.0 | 77.0 |
| **worm** | 53.0 | 76.0 | 87.0 | 92.0 |

# 2.4 ResNet Architecture

## 2.4.1 Introduction

The ResNet stands for Residual Network. This is a Neural Network Architecture designed to make training deep networks practical.
The core idea is Residual Block: **H(x) = x + F(x)**
**Where:**

- x: Input to block
- H(x): Desired mapping
- F(x): Residual function to be learned




The Residual Block

A **Skip Connection** is a direct path that bypasses one or more layers and feeds the input to a later point in the network.

In a traditional network, data flows through each layer in sequence. Every layer transforms the input and passes the result to the next one. Skip connections take the original input and add it directly to the output of a layer further down the block.

ResNet architecture is all about one thing: skip connections. That one idea solved two problems that had been holding deep networks back - vanishing gradients and the degradation problem - and made it practical to train networks at a depth that wasn't possible before.

### 2.4.1.1 Forward and Backward Propagation

***Forward Propagation***

In the ResNet-18 architecture, forward propagation transforms an input image batch $X \in \mathbb{R}^{(N\times3\times32\times32)}$ into target class logits $Z \in \mathbb{R}^{(N\times100)}$ through the following sequential operations:

- **Initial Stem Layer:**

    - Convolution: H_0 = Conv2d(X) with 64 filters of size 3 × 3, stride 1, and padding 1.
    - Batch Normalization & Activation: A_0 = ReLU(BatchNorm2d(H_0)).
- **Residual Layer Cascade:**

The output passes through 4 stacked residual layer groups (L=1, 2, 3, 4), each comprising 2 basic residual blocks. For block l with residual path F(x_l, {W_l}) and shortcut connection S(x_l):

$$x_{l+1} = \mathrm{ReLU}(F(x_l, \{W_l\}) + S(x_l))$$

where S(x_l) = x_l (identity) when dimensions match, or a 1 × 1 convolution with BatchNorm when spatial resolution is halved and feature channels are doubled.

- **Global Pooling & Classification Head:**
    - Spatial Aggregation: f = AdaptiveAvgPool2d(x_final) ∈ ℝ^(N×512×1×1), flattened to ℝ^(N×512).
    - Linear Head: Z = f W_fc + b_fc ∈ ℝ^(N×100).
    - Loss Computation: Cross-Entropy loss is computed between predicted logits Z and ground-truth labels y:

$$L = -1/N \sum_{i=1}^{N} \log( e^{Z_{i, y_i}} / \sum_{j=1}^{100} e^{Z_{i, j}} )$$

***Backward Propagation and Gradient Flow***

Backpropagation updates weight parameters by computing gradients via the chain rule. ResNet's skip connections fundamentally alter gradient flow:

- **Residual Gradient Formulation:**

For any deeper block output x_L relative to an earlier block output x_l, repeated forward recursive addition yields:

$$x_L = x_l + \sum_{k=l}^{L-1} F(x_k, \{W_k\})$$

Differentiating the loss L with respect to the earlier activation x_l gives:

$$\partial L / \partial x_l = (\partial L / \partial x_L) ( I + \partial/\partial x_l \sum_{k=l}^{L-1} F(x_k, \{W_k\}) )$$

- **Mitigation of Vanishing Gradients:**

The identity term I guarantees an unimpeded gradient highway directly from deeper layers to shallower layers. Even if weight gradients along the residual path ∂F / ∂x_l approach zero, ∂L / ∂x_l remains non-zero due to the additive factor (∂L / ∂x_L) I, preventing gradient vanishing.

- **Weight Updates:**

Gradients for weight parameters W_l are derived as ∂L / ∂W_l = (∂L / ∂x_{l+1}) (∂F_l / ∂W_l), and parameter updates are performed using the AdamW optimizer with weight decay:

$$W_l \leftarrow W_l - \eta \cdot ( \hat{m}_t / (\sqrt{(\hat{v}_t)} + \varepsilon) + \lambda W_l )$$

## 2.4.2 ResNet From-Scratch

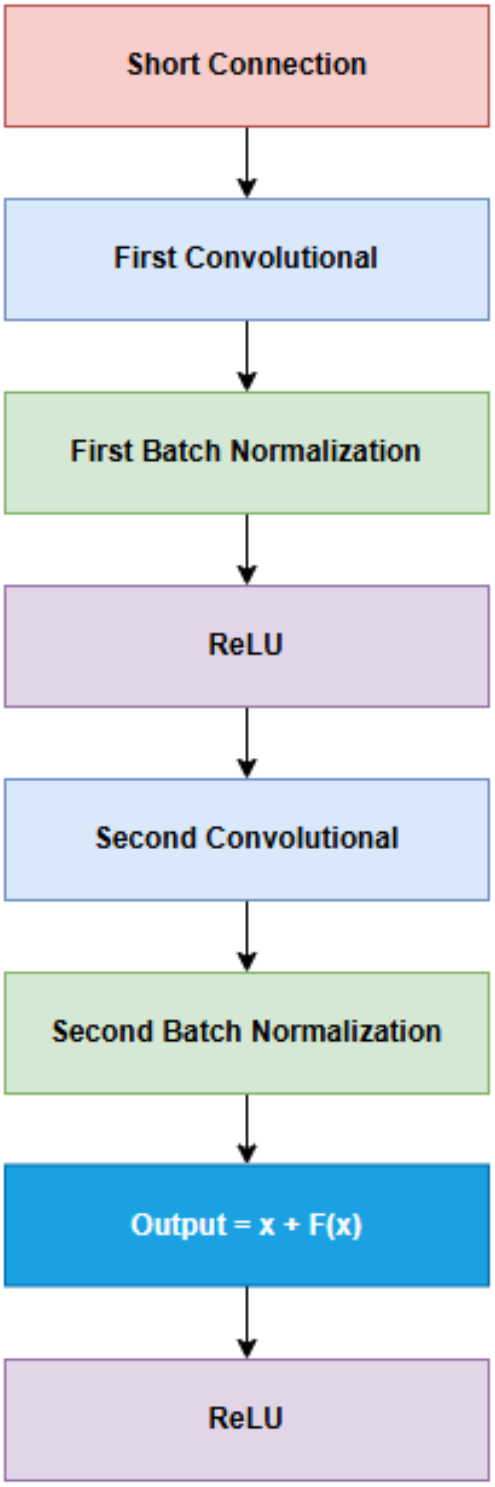
### 2.4.2.1 Basic Residual Block Implementation

#### 2.4.2.1.1 Basic Residual Block Configuration

- The function of Basic Residual Block is defined as **BasicBlock(nn.Module)**

| Layer | Input | Output |
|---|---|---|
| **First Convolutional:**<br>Conv2d(in_channels = <in_channels>, out_channels = <out_channels>, stride = <stride>, kernel_size = 3) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **First Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Second Convolutional:**<br>Conv2d(in_channels = <out_channels>, out_channels = <out_channels>, stride = 1, kernel_size = 3) | - tensor([N,<out_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout])) |
| **Second Batch Normalization:**<br>BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Short Connection:**<br>if(<sride>!= 1 OR <in_channels>!= <out_channels>)<br>⇒ Conv2d(in_channels = <in_channels>, out_channels = <out_channels>, stride = <stride>, kernel_size = 1)<br>⇒ BatchNorm2d(out_channels=<out_channels>)<br>else<br>⇒ Identity() | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |

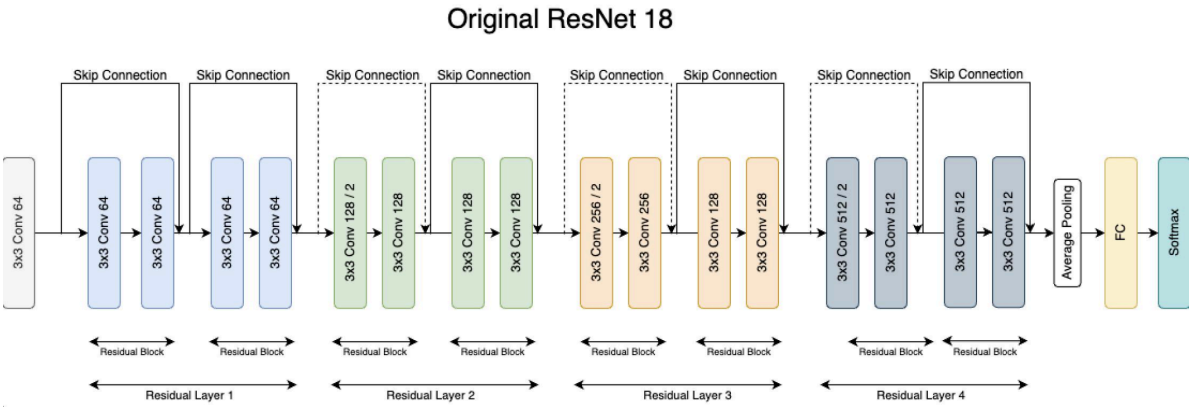
#### 2.4.2.1.2 Basic Residual Block Forward Implementation




*The Forward Implementation*

### 2.4.2.2 ResNet Implementation

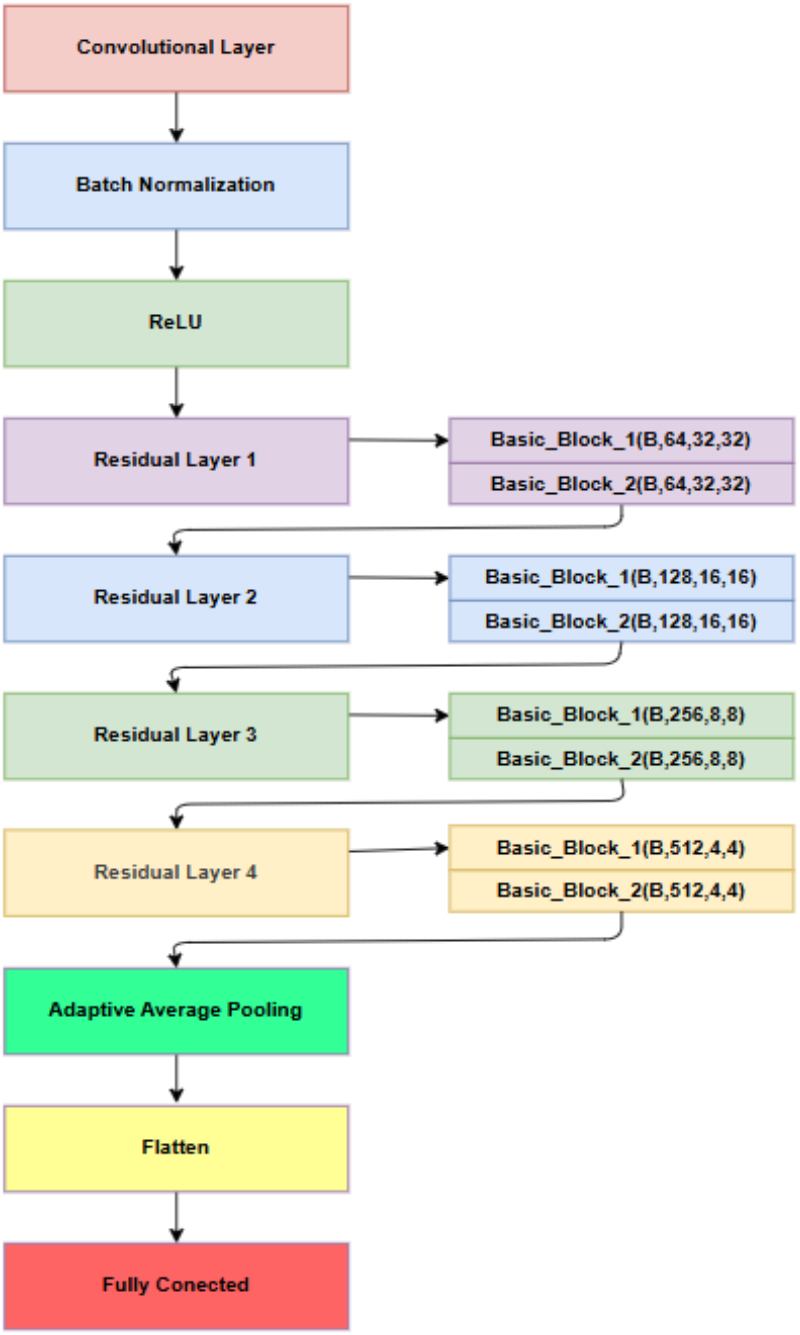
#### 2.4.2.2.1 ResNet Configuration




*ResNet-18 Architecture*

| Layer | Input | Output |
|---|---|---|
| **First Convolutional:** Conv2d(in_channels = 3, out_channels = 64, stride = <stride>, kernel_size = 3) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **First Batch Normalization:** BatchNorm2d(out_channels=<out_channels>) | - tensor([N,<in_channels>,Hin,Win]) | - tensor([N,<out_channels>,Hout,Wout]) |
| **Residual Block Layer 1** | - tensor(B,64,32,32) | - tensor(B,64,32,32) |
| **Residual Block Layer 2** | - tensor(B,64,32,32) | - tensor(B,128,16,16) |
| **Residual Block Layer 3** | - tensor(B,128,16,16) | - tensor(B,256,8,8) |
| **Residual Block Layer 4** | - tensor(B,256,8,8) | - tensor(B,512,4,4) |
| **Adaptive Average Pooling:** AdaptiveAvgPool2d((1,1)) | - tensor(B,512,4,4) | - tensor(B,512,1,1) |
| **Fully Connected:** Linear(512,100) | - tensor(B,512,1,1) | - tensor(B,100) |

#### 2.4.2.2.2 ResNet Forward Implementation




*The Resnet-18 Forward Function*

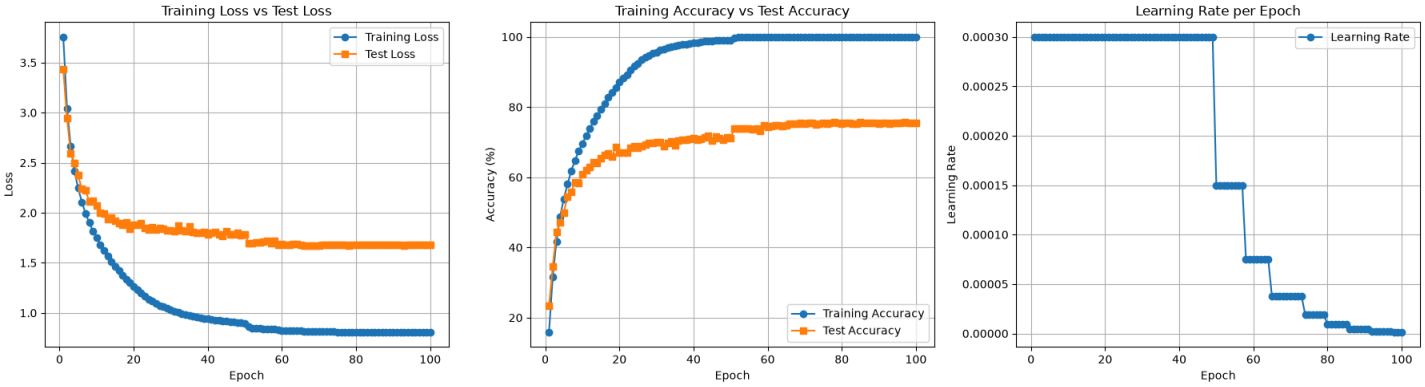
### 2.4.2.2 Discussion And Experimental Analysis

### 2.4.2.3 Analysis

- **Hyperparameters Configuration:**

| Epoch | Learning Rate | Batch Size | Train Loss | Test Loss | Train Accuracy | Test Accuracy | Cross Entropy Function |
|---|---|---|---|---|---|---|---|
| 100 | 0.0001 | 128 | 0.8137 | 1.7636 | 99.97% | 73.47% | AdamW(weight_ |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | | | | decay=1e-3) |




*The training progress of ResNet-18 From Scratch*

The table accuracy of **100 classes** based on the **Test Dataset**

| **Classes** | **Accuracy** |
|---|---|
| apples | 91.0% |
| aquarium fish | 85.0% |
| baby | 59.0% |
| bear | 60.0% |
| beaver | 67.0% |
| bed | 82.0% |
| bee | 87.0% |
| beetle | 79.0% |
| bicycle | 93.0% |
| bottles | 83.0% |
| bowls | 52.0% |
| boy | 50.0% |
| bridge | 87.0% |
| bus | 69.0% |
| butterfly | 70.0% |

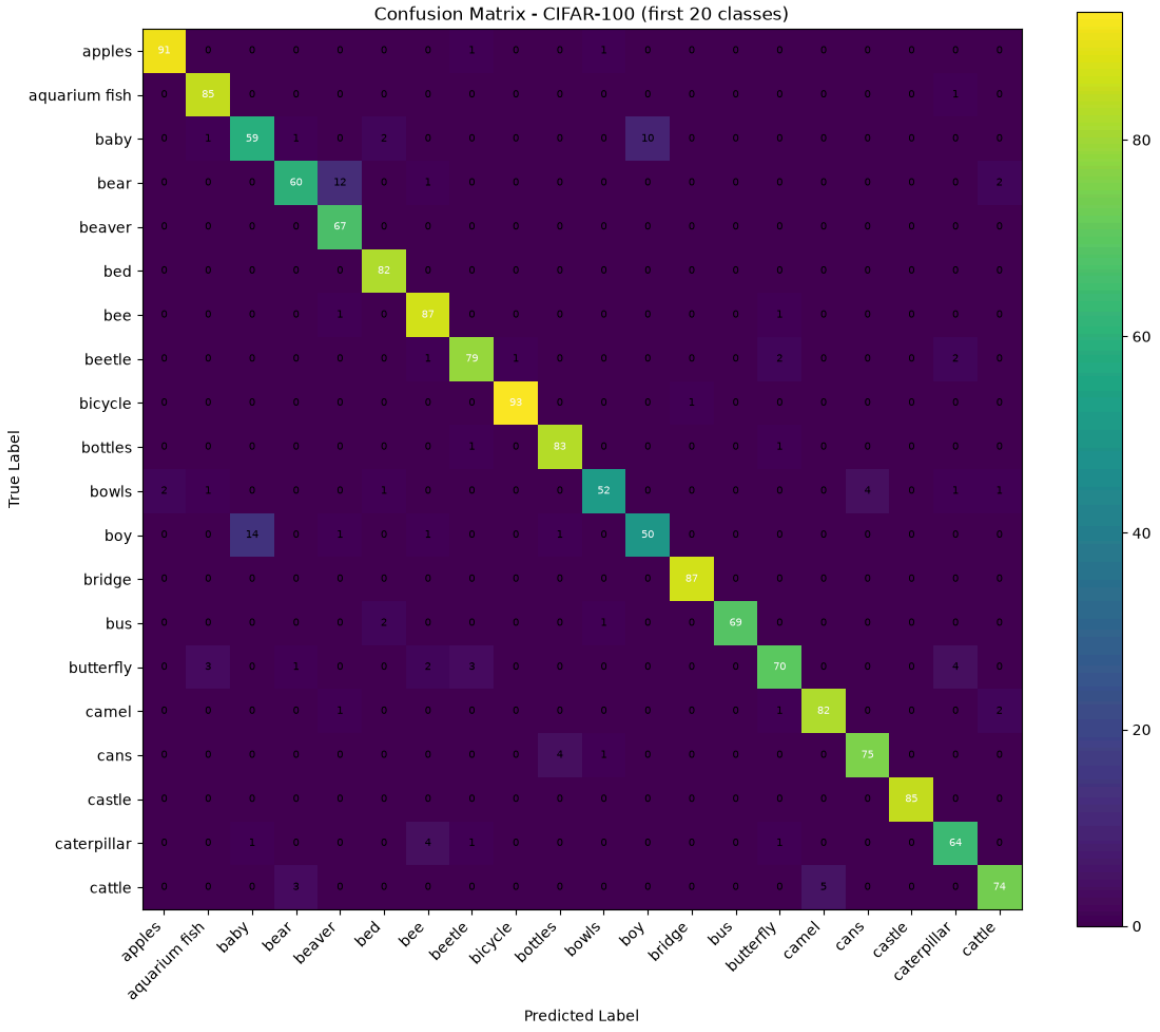
| | |
|---|---|
| camel | 82.0% |
| cans | 75.0% |
| castle | 85.0% |
| caterpillar | 64.0% |
| cattle | 74.0% |
| chair | 88.0% |
| chimpanzee | 92.0% |
| clock | 73.0% |
| cloud | 83.0% |
| cockroach | 87.0% |
| computer keyboard | 61.0% |
| couch | 68.0% |
| crab | 62.0% |
| crocodile | 85.0% |
| cups | 73.0% |
| dinosaur | 72.0% |
| dolphin | 74.0% |
| elephant | 65.0% |
| flatfish | 70.0% |
| forest | 82.0% |
| fox | 51.0% |
| girl | 85.0% |
| hamster | 82.0% |
| house | 69.0% |
| kangaroo | 89.0% |

| | |
|---|---|
| lamp | 68.0% |
| lawn-mower | 91.0% |
| leopard | 76.0% |
| lion | 83.0% |
| lizard | 58.0% |
| lobster | 68.0% |
| man | 56.0% |
| maple | 66.0% |
| motorcycle | 95.0% |
| mountain | 88.0% |
| mouse | 62.0% |
| mushrooms | 71.0% |
| oak | 67.0% |
| oranges | 91.0% |
| orchids | 89.0% |
| otter | 53.0% |
| palm | 90.0% |
| pears | 81.0% |
| pickup truck | 89.0% |
| pine | 74.0% |
| plain | 90.0% |
| plates | 70.0% |
| poppies | 78.0% |
| porcupine | 75.0% |
| possum | 58.0% |

| | |
|---|---|
| rabbit | 63.0% |
| raccoon | 82.0% |
| ray | 68.0% |
| road | 94.0% |
| rocket | 86.0% |
| roses | 78.0% |
| sea | 84.0% |
| seal | 45.0% |
| shark | 61.0% |
| shrew | 57.0% |
| skunk | 93.0% |
| skyscraper | 92.0% |
| snail | 69.0% |
| snake | 66.0% |
| spider | 84.0% |
| squirrel | 66.0% |
| streetcar | 79.0% |
| sunflowers | 91.0% |
| sweet peppers | 72.0% |
| table | 71.0% |
| tank | 85.0% |
| telephone | 72.0% |
| television | 85.0% |
| tiger | 81.0% |
| tractor | 92.0% |

| | |
|---|---|
| train | 86.0% |
| trout | 83.0% |
| tulips | 63.0% |
| turtle | 60.0% |
| wardrobe | 95.0% |
| whale | 73.0% |
| willow | 71.0% |
| wolf | 79.0% |
| woman | 63.0% |
| worm | 68.0% |

**Confusion Matrix of 20 Classes**



**Classification Metric**

| Classes | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| apples | 0.90 | 0.87 | 0.88 | 100 |
| aquarium fish | 0.81 | 0.87 | 0.84 | 100 |
| baby | 0.66 | 0.63 | 0.65 | 100 |
| bear | 0.61 | 0.57 | 0.59 | 100 |
| beaver | 0.58 | 0.64 | 0.61 | 100 |
| bed | 0.78 | 0.77 | 0.77 | 100 |
| bee | 0.72 | 0.81 | 0.76 | 100 |
| beetle | 0.67 | 0.76 | 0.71 | 100 |
| bicycle | 0.86 | 0.87 | 0.87 | 100 |
| bottles | 0.80 | 0.85 | 0.83 | 100 |
| bowls | 0.62 | 0.51 | 0.56 | 100 |
| boy | 0.54 | 0.50 | 0.52 | 100 |
| bridge | 0.77 | 0.81 | 0.79 | 100 |
| bus | 0.70 | 0.67 | 0.68 | 100 |
| butterfly | 0.73 | 0.66 | 0.69 | 100 |
| camel | 0.72 | 0.77 | 0.74 | 100 |
| cans | 0.77 | 0.73 | 0.75 | 100 |
| castle | 0.89 | 0.84 | 0.87 | 100 |
| caterpillar | 0.71 | 0.63 | 0.67 | 100 |
| cattle | 0.76 | 0.65 | 0.70 | 100 |
| chair | 0.82 | 0.87 | 0.84 | 100 |
| chimpanzee | 0.84 | 0.88 | 0.86 | 100 |
| clock | 0.73 | 0.74 | 0.73 | 100 |

| | | | | |
|---|---|---|---|---|
| cloud | 0.84 | 0.83 | 0.83 | 100 |
| cockroach | 0.85 | 0.83 | 0.84 | 100 |
| computer keyboard | 0.64 | 0.64 | 0.64 | 100 |
| couch | 0.65 | 0.67 | 0.66 | 100 |
| crab | 0.65 | 0.57 | 0.61 | 100 |
| crocodile | 0.85 | 0.82 | 0.83 | 100 |
| cups | 0.80 | 0.73 | 0.76 | 100 |
| dinosaur | 0.72 | 0.68 | 0.70 | 100 |
| dolphin | 0.69 | 0.68 | 0.68 | 100 |
| elephant | 0.71 | 0.67 | 0.69 | 100 |
| flatfish | 0.71 | 0.60 | 0.65 | 100 |
| forest | 0.72 | 0.81 | 0.76 | 100 |
| fox | 0.52 | 0.52 | 0.52 | 100 |
| girl | 0.80 | 0.82 | 0.81 | 100 |
| hamster | 0.75 | 0.80 | 0.77 | 100 |
| house | 0.66 | 0.63 | 0.64 | 100 |
| kangaroo | 0.91 | 0.85 | 0.88 | 100 |
| lamp | 0.66 | 0.62 | 0.64 | 100 |
| lawn-mower | 0.89 | 0.88 | 0.88 | 100 |
| leopard | 0.71 | 0.75 | 0.73 | 100 |
| lion | 0.85 | 0.86 | 0.86 | 100 |
| lizard | 0.51 | 0.52 | 0.51 | 100 |
| lobster | 0.61 | 0.61 | 0.61 | 100 |
| man | 0.58 | 0.53 | 0.55 | 100 |
| maple | 0.67 | 0.69 | 0.68 | 100 |

| | | | | |
|---|---|---|---|---|
| motorcycle | 0.84 | 0.94 | 0.89 | 100 |
| mountain | 0.80 | 0.90 | 0.85 | 100 |
| mouse | 0.56 | 0.58 | 0.57 | 100 |
| mushrooms | 0.76 | 0.75 | 0.75 | 100 |
| oak | 0.63 | 0.69 | 0.66 | 100 |
| oranges | 0.85 | 0.94 | 0.90 | 100 |
| orchids | 0.79 | 0.87 | 0.83 | 100 |
| otter | 0.42 | 0.42 | 0.42 | 100 |
| palm | 0.89 | 0.91 | 0.90 | 100 |
| pears | 0.77 | 0.77 | 0.77 | 100 |
| pickup truck | 0.84 | 0.85 | 0.85 | 100 |
| pine | 0.73 | 0.67 | 0.70 | 100 |
| plain | 0.86 | 0.89 | 0.87 | 100 |
| plates | 0.69 | 0.70 | 0.70 | 100 |
| poppies | 0.78 | 0.80 | 0.79 | 100 |
| porcupine | 0.74 | 0.67 | 0.71 | 100 |
| possum | 0.71 | 0.58 | 0.64 | 100 |
| rabbit | 0.62 | 0.60 | 0.61 | 100 |
| raccoon | 0.81 | 0.83 | 0.82 | 100 |
| ray | 0.64 | 0.61 | 0.62 | 100 |
| road | 0.85 | 0.94 | 0.90 | 100 |
| rocket | 0.83 | 0.85 | 0.84 | 100 |
| roses | 0.75 | 0.79 | 0.77 | 100 |
| sea | 0.78 | 0.87 | 0.82 | 100 |
| seal | 0.49 | 0.49 | 0.49 | 100 |

| | | | | |
|---|---|---|---|---|
| shark | 0.60 | 0.58 | 0.59 | 100 |
| shrew | 0.55 | 0.54 | 0.54 | 100 |
| skunk | 0.88 | 0.95 | 0.91 | 100 |
| skyscraper | 0.87 | 0.92 | 0.89 | 100 |
| snail | 0.69 | 0.64 | 0.66 | 100 |
| snake | 0.66 | 0.61 | 0.63 | 100 |
| spider | 0.79 | 0.82 | 0.80 | 100 |
| squirrel | 0.52 | 0.65 | 0.58 | 100 |
| streetcar | 0.72 | 0.75 | 0.74 | 100 |
| sunflowers | 0.92 | 0.93 | 0.93 | 100 |
| sweet peppers | 0.78 | 0.72 | 0.75 | 100 |
| table | 0.77 | 0.67 | 0.72 | 100 |
| tank | 0.82 | 0.84 | 0.83 | 100 |
| telephone | 0.80 | 0.74 | 0.77 | 100 |
| television | 0.76 | 0.83 | 0.79 | 100 |
| tiger | 0.87 | 0.80 | 0.83 | 100 |
| tractor | 0.73 | 0.85 | 0.78 | 100 |
| train | 0.75 | 0.83 | 0.79 | 100 |
| trout | 0.78 | 0.86 | 0.82 | 100 |
| tulips | 0.69 | 0.59 | 0.63 | 100 |
| turtle | 0.72 | 0.58 | 0.64 | 100 |
| wardrobe | 0.85 | 0.92 | 0.88 | 100 |
| whale | 0.74 | 0.75 | 0.74 | 100 |
| willow | 0.68 | 0.63 | 0.66 | 100 |
| wolf | 0.80 | 0.82 | 0.81 | 100 |

| | | | | |
|---|---|---|---|---|
| woman | 0.53 | 0.53 | 0.53 | 100 |
| worm | 0.78 | 0.70 | 0.74 | 100 |

### 2.4.3 ResNet Pretrained

#### 2.4.3.1 ResNet Pretrained Introduction

A pretrained model is a machine learning model that has been previously trained on a large dataset for a specific task (usually general-purpose) and can then be reused or fine-tuned for a different but related task. Pre-trained models save development teams time, data and computational resources compared to training a model from scratch.
There are common three type of Pretrained Model: Freeze All Of Backbone, Freeze Partial Backbone, and Full Finetune

#### 2.4.3.2 ResNet Pretrained Loading Model

The model which is used is ResNet-18 with CIFAR-100 dataset from Hugging Face as following the link -
**https://huggingface.co/edadaltocg/resnet18_cifar100/resolve/main/pytorch_model.bin**

#### 2.4.3.3 The Analysis Of Pretrained Model

| **Layer 1** | |
|---|---|
| **Basic Block 1** | **First Convolutional:**<br>Conv2d(in_channels = 64, out_channels = 64, stride = 1, kernel_size = 3)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=64)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 64, out_channels = 64, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=64)<br>**Activation:**<br>ReLU(inplace= True) |
| **Basic Block 2** | **First Convolutional:**<br>Conv2d(in_channels = 64, out_channels = 64, stride = 1, kernel_size = 3)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=64) |

| | |
|---|---|
| | **Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 64, out_channels = 64, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=64)<br>**Activation:**<br>ReLU(inplace= True) |
| **Layer 2** | |
| **Basic Block 1** | **First Convolutional:**<br>Conv2d(in_channels = 64, out_channels = 128, stride = 2, kernel_size = 3, padding =1)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=128)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 128, out_channels = 128, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=128)<br>**Activation:**<br>ReLU(inplace= True)<br>**Down Sample:**<br>Conv2d(64, 128, kernel_size=1, stride=2, bias=False)<br>BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) |
| **Basic Block 2** | **First Convolutional:**<br>Conv2d(in_channels = 128, out_channels = 128, stride = 1, kernel_size = 3)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=128)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 128, out_channels = 128, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=128) |

| | |
|---|---|
| | **Activation:**<br>ReLU(inplace= True) |

| **Layer 3** | |
|---|---|
| **Basic Block 1** | **First Convolutional:**<br>Conv2d(in_channels = 128, out_channels = 256, stride = 2, kernel_size = 3, padding =1)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=256)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 256, out_channels = 256, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=256)<br>**Activation:**<br>ReLU(inplace= True)<br>**Down Sample:**<br>Conv2d(128, 256, kernel_size=1, stride=2, bias=False)<br>BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) |
| **Basic Block 2** | **First Convolutional:**<br>Conv2d(in_channels = 256, out_channels = 256, stride = 1, kernel_size = 3)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=256)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 256, out_channels = 256, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=256)<br>**Activation:**<br>ReLU(inplace= True) |

| **Layer 4** | |
|---|---|

| | |
|---|---|
| **Basic Block 1** | **First Convolutional:**<br>Conv2d(in_channels = 256, out_channels = 512, stride = 2, kernel_size = 3, padding =1)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=512)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 512, out_channels = 512, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=512)<br>**Activation:**<br>ReLU(inplace= True)<br>**Down Sample:**<br>Conv2d(256, 512, kernel_size=1, stride=2, bias=False)<br>BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) |
| **Basic Block 2** | **First Convolutional:**<br>Conv2d(in_channels = 512, out_channels = 512, stride = 1, kernel_size = 3)<br>**First Batch Normalization:**<br>BatchNorm2d(out_channels=512)<br>**Activation:**<br>ReLU(inplace= True)<br>**Second Convolutional:**<br>Conv2d(in_channels = 512, out_channels = 512, stride = 1, kernel_size = 3)<br>**Second Batch Normalization:**<br>BatchNorm2d(out_channels=512)<br>**Activation:**<br>ReLU(inplace= True) |

| **Main Layers** |
|---|
| **Convolutional Layer:**<br>Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False) |
| **Batch Normalization:**<br>BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True) |

| |
|---|
| **Activation:**<br>ReLU(inplace=True) |
| **MaxPooling:**<br>Identity() |
| **Layer 1** |
| **Layer 2** |
| **Layer 3** |
| **Layer 4** |
| **Global Pool:**<br>SelectAdaptivePool2d(pool_type=avg, flatten=Flatten(start_dim=1, end_dim=-1)) |
| **Fully Connected:**<br>Linear(in_features=512, out_features=100, bias=True) |

#### 2.4.3.1 ResNet Pretrained Types

##### 2.4.3.1.1 ResNet Pretrained - Freeze All Of Backbone

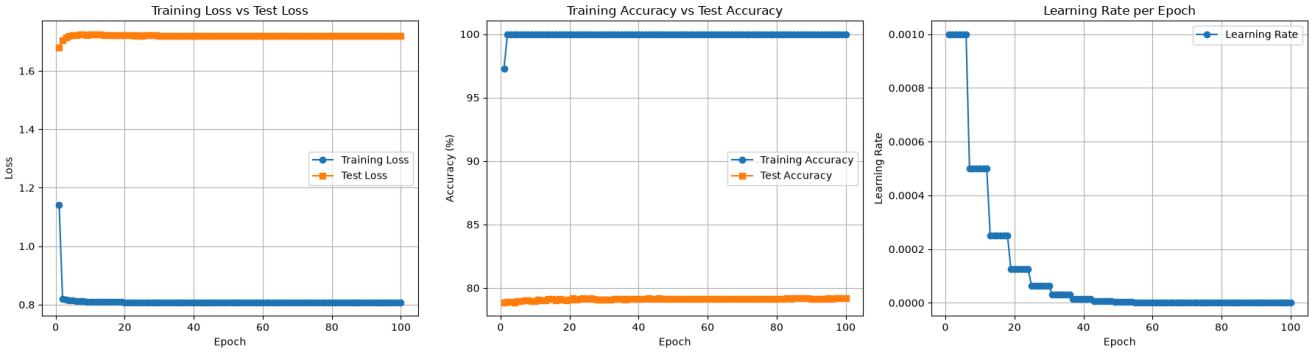
Freezing all of backbone means locking its weights so they do not update during training, turning the backbone into a fixed feature extractor while only training a new or replacement head.

| | |
|---|---|
| **Freeze layers** | conv1, bn1, layer1, layer2, layer3, layer4 |
| **Unfreeze layers** | fully connected |

*The table of Pretrained - Freeze All Of Backbone Strategy*

- **Hyperparameters Configuration:**

| Epoch | Learning Rate | Batch Size | Train Loss | Test Loss | Train Accuracy | Test Accuracy | Cross Entropy Function | Trainable Parameter |
|---|---|---|---|---|---|---|---|---|
| 100 | 0.001 | 128 | 0.8077 | 1.7202 | 99.98% | 79.19% | AdamW(weight_decay=1e-3) | 51300 |




*The training progress of ResNet-18 Pretrained Freezing All of Backbone*

The table accuracy of **100 classes** based on the **Test Dataset**

| **Classes** | **Accuracy** |
|---|---|
| apples | 94.0% |
| aquarium fish | 93.0% |
| baby | 74.0% |
| bear | 68.0% |
| beaver | 71.0% |
| bed | 81.0% |
| bee | 86.0% |
| beetle | 79.0% |
| bicycle | 92.0% |
| bottles | 92.0% |
| bowls | 55.0% |
| boy | 54.0% |
| bridge | 86.0% |
| bus | 68.0% |
| butterfly | 78.0% |
| camel | 85.0% |
| cans | 79.0% |

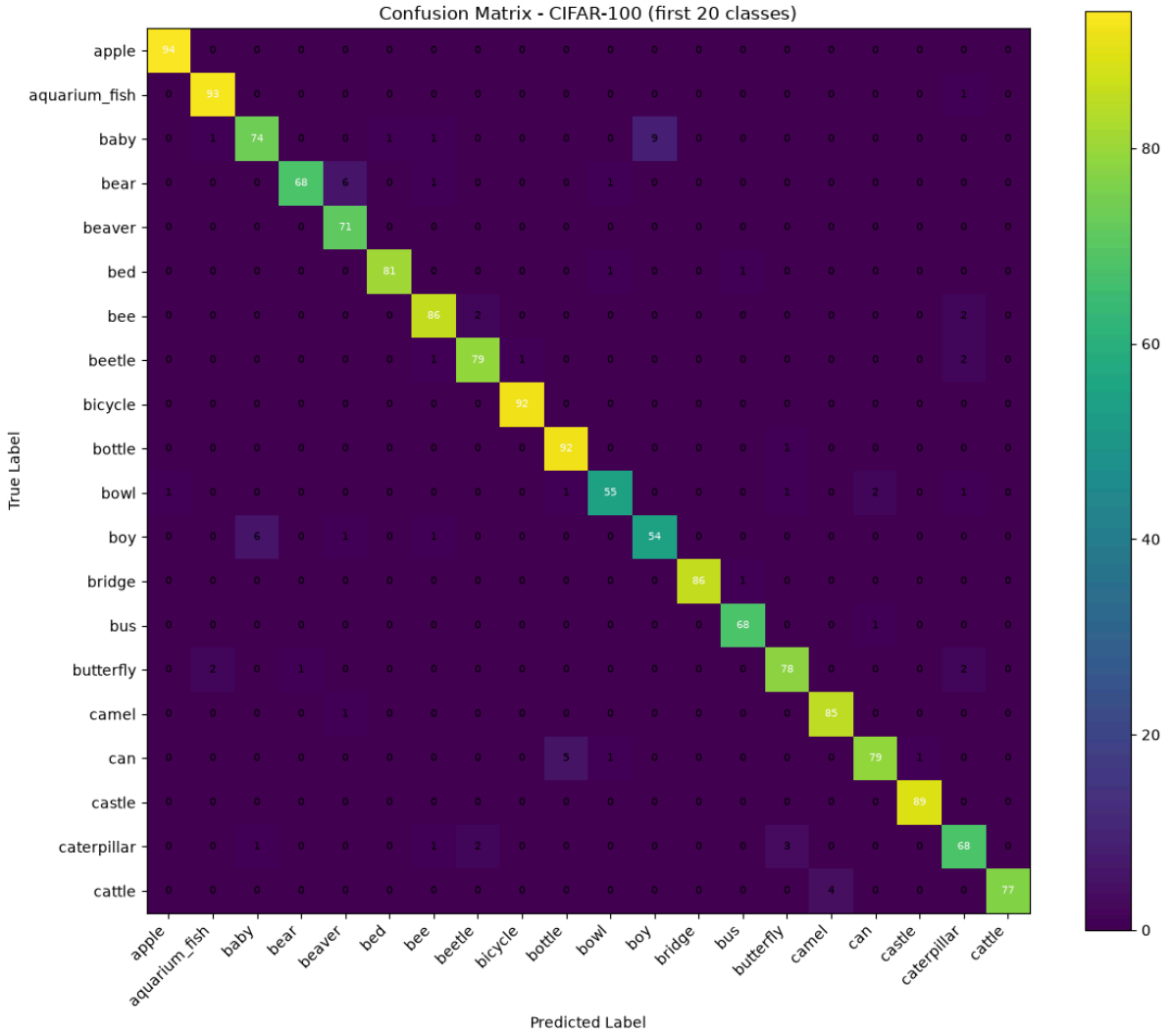
| | |
|---|---|
| castle | 89.0% |
| caterpillar | 68.0% |
| cattle | 77.0% |
| chair | 90.0% |
| chimpanzee | 91.0% |
| clock | 82.0% |
| cloud | 79.0% |
| cockroach | 86.0% |
| computer keyboard | 67.0% |
| couch | 77.0% |
| crab | 71.0% |
| crocodile | 85.0% |
| cups | 80.0% |
| dinosaur | 78.0% |
| dolphin | 73.0% |
| elephant | 77.0% |
| flatfish | 66.0% |
| forest | 87.0% |
| fox | 58.0% |
| girl | 86.0% |
| hamster | 79.0% |
| house | 80.0% |
| kangaroo | 91.0% |
| lamp | 79.0% |
| lawn-mower | 92.0% |

| | |
|---|---|
| leopard | 78.0% |
| lion | 89.0% |
| lizard | 58.0% |
| lobster | 74.0% |
| man | 54.0% |
| maple | 74.0% |
| motorcycle | 96.0% |
| mountain | 92.0% |
| mouse | 63.0% |
| mushrooms | 79.0% |
| oak | 76.0% |
| oranges | 96.0% |
| orchids | 87.0% |
| otter | 58.0% |
| palm | 94.0% |
| pears | 85.0% |
| pickup truck | 89.0% |
| pine | 72.0% |
| plain | 92.0% |
| plates | 73.0% |
| poppies | 82.0% |
| porcupine | 78.0% |
| possum | 61.0% |
| rabbit | 69.0% |
| raccoon | 87.0% |

| | |
|---|---|
| ray | 69.0% |
| road | 97.0% |
| rocket | 87.0% |
| roses | 81.0% |
| sea | 88.0% |
| seal | 51.0% |
| shark | 59.0% |
| shrew | 60.0% |
| skunk | 94.0% |
| skyscraper | 94.0% |
| snail | 76.0% |
| snake | 69.0% |
| spider | 86.0% |
| squirrel | 77.0% |
| streetcar | 84.0% |
| sunflowers | 95.0% |
| sweet peppers | 75.0% |
| table | 77.0% |
| tank | 93.0% |
| telephone | 79.0% |
| television | 87.0% |
| tiger | 90.0% |
| tractor | 95.0% |
| train | 88.0% |
| trout | 86.0% |

| | |
|---|---|
| tulips | 70.0% |
| turtle | 67.0% |
| wardrobe | 96.0% |
| whale | 76.0% |
| willow | 69.0% |
| wolf | 85.0% |
| woman | 63.0% |
| worm | 77.0% |

**Confusion Matrix of 20 Classes**




**Classification Metric**

| Accuracy | Precision | Recall | F1-score |
|---|---|---|---|

| | | | |
|---|---|---|---|
| 79.19% | 0.7917 | 0.7919 | 0.7908 |

| Classes | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| apple | 0.91 | 0.94 | 0.93 | 100 |
| aquarium_fish | 0.89 | 0.93 | 0.91 | 100 |
| baby | 0.73 | 0.74 | 0.74 | 100 |
| bear | 0.74 | 0.68 | 0.71 | 100 |
| beaver | 0.66 | 0.71 | 0.68 | 100 |
| bed | 0.82 | 0.81 | 0.81 | 100 |
| bee | 0.89 | 0.86 | 0.87 | 100 |
| beetle | 0.82 | 0.79 | 0.81 | 100 |
| bicycle | 0.96 | 0.92 | 0.94 | 100 |
| bottle | 0.86 | 0.92 | 0.89 | 100 |
| bowl | 0.71 | 0.55 | 0.62 | 100 |
| boy | 0.57 | 0.54 | 0.55 | 100 |
| bridge | 0.88 | 0.86 | 0.87 | 100 |
| bus | 0.80 | 0.68 | 0.74 | 100 |
| butterfly | 0.84 | 0.78 | 0.81 | 100 |
| camel | 0.75 | 0.85 | 0.80 | 100 |
| can | 0.84 | 0.79 | 0.81 | 100 |
| castle | 0.93 | 0.89 | 0.91 | 100 |
| caterpillar | 0.70 | 0.68 | 0.69 | 100 |
| cattle | 0.83 | 0.77 | 0.80 | 100 |
| chair | 0.91 | 0.90 | 0.90 | 100 |
| chimpanzee | 0.86 | 0.91 | 0.88 | 100 |
| clock | 0.83 | 0.82 | 0.82 | 100 |

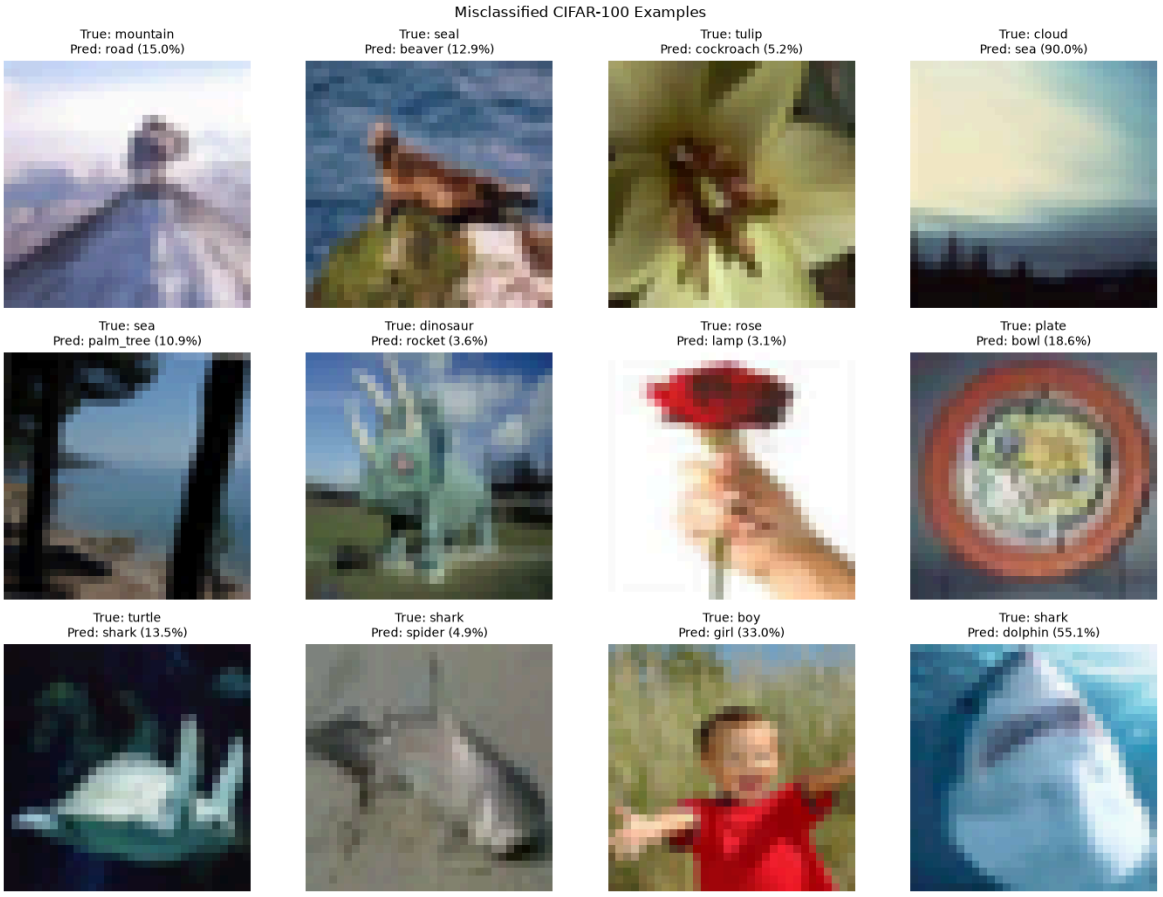
| | | | | |
|---|---|---|---|---|
| cloud | 0.85 | 0.79 | 0.82 | 100 |
| cockroach | 0.85 | 0.86 | 0.86 | 100 |
| couch | 0.71 | 0.67 | 0.69 | 100 |
| crab | 0.74 | 0.77 | 0.75 | 100 |
| crocodile | 0.69 | 0.71 | 0.70 | 100 |
| cup | 0.87 | 0.85 | 0.86 | 100 |
| dinosaur | 0.87 | 0.80 | 0.83 | 100 |
| dolphin | 0.70 | 0.78 | 0.74 | 100 |
| elephant | 0.77 | 0.73 | 0.75 | 100 |
| flatfish | 0.82 | 0.77 | 0.79 | 100 |
| forest | 0.73 | 0.66 | 0.69 | 100 |
| fox | 0.85 | 0.87 | 0.86 | 100 |
| girl | 0.59 | 0.58 | 0.58 | 100 |
| hamster | 0.87 | 0.86 | 0.86 | 100 |
| house | 0.80 | 0.79 | 0.79 | 100 |
| kangaroo | 0.73 | 0.80 | 0.77 | 100 |
| keyboard | 0.95 | 0.91 | 0.93 | 100 |
| lamp | 0.82 | 0.79 | 0.81 | 100 |
| lawn_mower | 0.94 | 0.92 | 0.93 | 100 |
| leopard | 0.77 | 0.78 | 0.78 | 100 |
| lion | 0.84 | 0.89 | 0.86 | 100 |
| lizard | 0.59 | 0.58 | 0.58 | 100 |
| lobster | 0.70 | 0.74 | 0.72 | 100 |
| man | 0.57 | 0.54 | 0.56 | 100 |
| maple_tree | 0.70 | 0.74 | 0.72 | 100 |

| | | | | |
|---|---|---|---|---|
| motorcycle | 0.91 | 0.96 | 0.93 | 100 |
| mountain | 0.84 | 0.92 | 0.88 | 100 |
| mouse | 0.68 | 0.63 | 0.66 | 100 |
| mushroom | 0.87 | 0.79 | 0.83 | 100 |
| oak_tree | 0.66 | 0.76 | 0.70 | 100 |
| orange | 0.90 | 0.96 | 0.93 | 100 |
| orchid | 0.82 | 0.87 | 0.84 | 100 |
| otter | 0.61 | 0.58 | 0.59 | 100 |
| palm_tree | 0.92 | 0.94 | 0.93 | 100 |
| pear | 0.86 | 0.85 | 0.85 | 100 |
| pickup_truck | 0.92 | 0.89 | 0.90 | 100 |
| pine_tree | 0.77 | 0.72 | 0.74 | 100 |
| plain | 0.86 | 0.92 | 0.89 | 100 |
| plate | 0.82 | 0.73 | 0.77 | 100 |
| poppy | 0.77 | 0.82 | 0.79 | 100 |
| porcupine | 0.80 | 0.78 | 0.79 | 100 |
| possum | 0.73 | 0.61 | 0.67 | 100 |
| rabbit | 0.72 | 0.69 | 0.70 | 100 |
| raccoon | 0.86 | 0.87 | 0.87 | 100 |
| ray | 0.75 | 0.69 | 0.72 | 100 |
| road | 0.87 | 0.97 | 0.92 | 100 |
| rocket | 0.86 | 0.87 | 0.87 | 100 |
| rose | 0.78 | 0.81 | 0.79 | 100 |
| sea | 0.81 | 0.88 | 0.84 | 100 |
| seal | 0.50 | 0.51 | 0.50 | 100 |

| | | | | |
|---|---|---|---|---|
| shark | 0.63 | 0.59 | 0.61 | 100 |
| shrew | 0.58 | 0.60 | 0.59 | 100 |
| skunk | 0.91 | 0.94 | 0.93 | 100 |
| skyscraper | 0.92 | 0.94 | 0.93 | 100 |
| snail | 0.78 | 0.76 | 0.77 | 100 |
| snake | 0.68 | 0.69 | 0.68 | 100 |
| spider | 0.77 | 0.86 | 0.81 | 100 |
| squirrel | 0.71 | 0.77 | 0.74 | 100 |
| streetcar | 0.73 | 0.84 | 0.78 | 100 |
| sunflower | 0.94 | 0.95 | 0.95 | 100 |
| sweet_pepper | 0.76 | 0.75 | 0.75 | 100 |
| table | 0.83 | 0.77 | 0.80 | 100 |
| tank | 0.90 | 0.93 | 0.92 | 100 |
| telephone | 0.87 | 0.79 | 0.83 | 100 |
| television | 0.78 | 0.87 | 0.82 | 100 |
| tiger | 0.87 | 0.90 | 0.89 | 100 |
| tractor | 0.86 | 0.95 | 0.90 | 100 |
| train | 0.81 | 0.88 | 0.84 | 100 |
| trout | 0.79 | 0.86 | 0.82 | 100 |
| tulip | 0.76 | 0.70 | 0.73 | 100 |
| turtle | 0.81 | 0.67 | 0.73 | 100 |
| wardrobe | 0.86 | 0.96 | 0.91 | 100 |
| whale | 0.78 | 0.76 | 0.77 | 100 |
| willow_tree | 0.71 | 0.69 | 0.70 | 100 |
| wolf | 0.84 | 0.85 | 0.85 | 100 |

| | | | | |
|---|---|---|---|---|
| woman | 0.62 | 0.63 | 0.62 | 100 |
| worm | 0.76 | 0.77 | 0.77 | 100 |

**Display Misclassified Images based on 12 Classes:**




#### 2.4.3.1.2 ResNet Pretrained - Freeze Partial Backbone

Freezing a partial backbone of a pre-trained deep learning model means keeping some layers fixed (no gradient updates) while allowing other layers in the backbone or head to train.

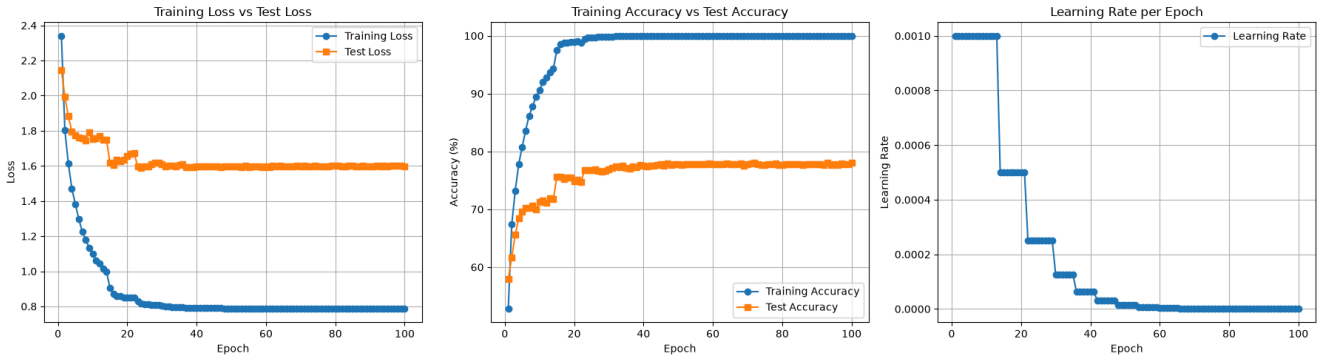
| | |
|---|---|
| **Freeze layers** | conv1, bn1, layer1 |
| **Unfreeze layers** | layer2, layer3, layer4, fully connected |

*The table of Pretrained - Freeze Partial Backbone Strategy*

- **Hyperparameters Configuration:**

| Epoch | Learning Rate | Batch Size | Train Loss | Test Loss | Train Accuracy | Test Accuracy | Cross Entropy Function | Trainable Parameter |
|---|---|---|---|---|---|---|---|---|
| 100 | 0.001 | 128 | 0.7867 | 1.5979 | 99.98% | 78.01% | AdamW(weight_decay=1e- | 11070308 |

| | | | | | | | 3) | |
|---|---|---|---|---|---|---|---|---|




*The training progress of ResNet-18 Pretrained Freezing Partial Backbone*

The table accuracy of **100 classes** based on the **Test Dataset**

| **Classes** | **Accuracy** |
|---|---|
| apple | 94.0% |
| aquarium_fish | 92.0% |
| baby | 67.0% |
| bear | 67.0% |
| beaver | 68.0% |
| bed | 79.0% |
| bee | 79.0% |
| beetle | 78.0% |
| bicycle | 92.0% |
| bottle | 90.0% |
| bowl | 61.0% |
| boy | 58.0% |
| bridge | 83.0% |
| bus | 70.0% |
| butterfly | 77.0% |

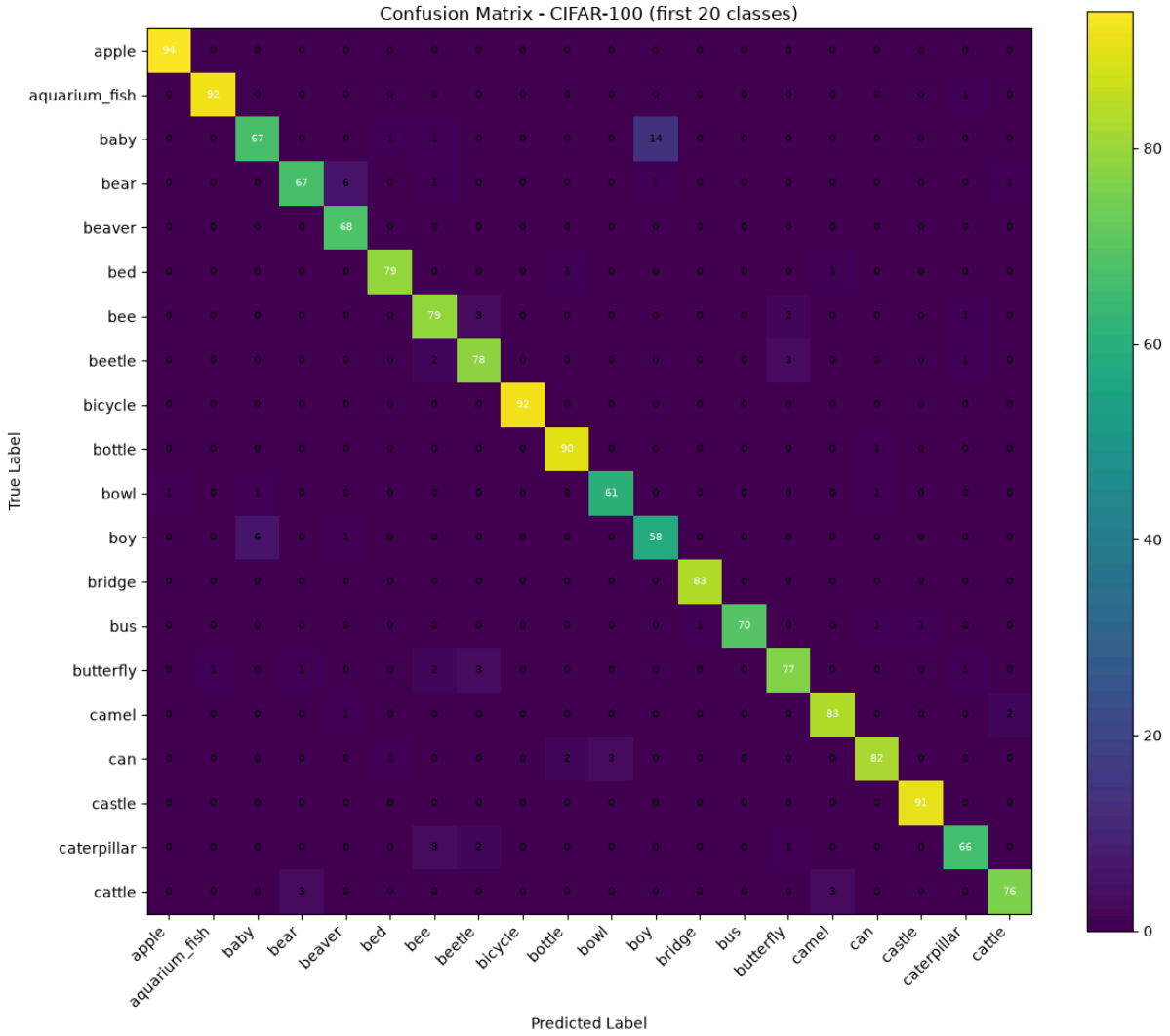
| | |
|---|---|
| camel | 83.0% |
| can | 82.0% |
| castle | 91.0% |
| caterpillar | 66.0% |
| cattle | 76.0% |
| chair | 87.0% |
| chimpanzee | 94.0% |
| clock | 81.0% |
| cloud | 84.0% |
| cockroach | 82.0% |
| couch | 72.0% |
| crab | 81.0% |
| crocodile | 68.0% |
| cup | 84.0% |
| dinosaur | 75.0% |
| dolphin | 76.0% |
| elephant | 73.0% |
| flatfish | 70.0% |
| forest | 67.0% |
| fox | 86.0% |
| girl | 55.0% |
| hamster | 84.0% |
| house | 77.0% |
| kangaroo | 75.0% |
| keyboard | 90.0% |

| | |
|---|---|
| lamp | 77.0% |
| lawn_mower | 89.0% |
| leopard | 82.0% |
| lion | 87.0% |
| lizard | 63.0% |
| lobster | 70.0% |
| man | 64.0% |
| maple_tree | 69.0% |
| motorcycle | 97.0% |
| mountain | 86.0% |
| mouse | 67.0% |
| mushroom | 75.0% |
| oak_tree | 67.0% |
| orange | 96.0% |
| orchid | 90.0% |
| otter | 63.0% |
| palm_tree | 90.0% |
| pear | 80.0% |
| pickup_truck | 92.0% |
| pine_tree | 70.0% |
| plain | 93.0% |
| plate | 75.0% |
| poppy | 77.0% |
| porcupine | 78.0% |
| possum | 61.0% |

| | |
|---|---|
| rabbit | 66.0% |
| raccoon | 88.0% |
| ray | 63.0% |
| road | 95.0% |
| rocket | 84.0% |
| rose | 77.0% |
| sea | 86.0% |
| seal | 54.0% |
| shark | 62.0% |
| shrew | 63.0% |
| skunk | 92.0% |
| skyscraper | 91.0% |
| snail | 72.0% |
| snake | 67.0% |
| spider | 80.0% |
| squirrel | 69.0% |
| streetcar | 83.0% |
| sunflower | 91.0% |
| sweet_pepper | 72.0% |
| table | 77.0% |
| tank | 90.0% |
| telephone | 77.0% |
| television | 87.0% |
| tiger | 84.0% |
| tractor | 92.0% |

| | |
|---|---|
| train | 91.0% |
| trout | 84.0% |
| tulip | 72.0% |
| turtle | 64.0% |
| wardrobe | 95.0% |
| whale | 76.0% |
| willow_tree | 66.0% |
| wolf | 86.0% |
| woman | 59.0% |
| worm | 74.0% |

**Confusion Matrix of 20 Classes**



**Classification Metric**

| Accuracy | Precision | Recall | F1-score |
|---|---|---|---|
| 78.01% | 0.7820 | 0.7801 | 0.7801 |

| Classes | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| apple | 0.93 | 0.94 | 0.94 | 100 |
| aquarium_fish | 0.93 | 0.92 | 0.92 | 100 |
| baby | 0.68 | 0.67 | 0.67 | 100 |
| bear | 0.71 | 0.67 | 0.69 | 100 |
| beaver | 0.69 | 0.68 | 0.69 | 100 |
| bed | 0.82 | 0.79 | 0.81 | 100 |
| bee | 0.80 | 0.79 | 0.79 | 100 |
| beetle | 0.75 | 0.78 | 0.76 | 100 |
| bicycle | 0.94 | 0.92 | 0.93 | 100 |
| bottle | 0.87 | 0.90 | 0.89 | 100 |
| bowl | 0.72 | 0.61 | 0.66 | 100 |
| boy | 0.54 | 0.58 | 0.56 | 100 |
| bridge | 0.82 | 0.83 | 0.83 | 100 |
| bus | 0.79 | 0.70 | 0.74 | 100 |
| butterfly | 0.76 | 0.77 | 0.77 | 100 |
| camel | 0.75 | 0.83 | 0.79 | 100 |
| can | 0.79 | 0.82 | 0.80 | 100 |
| castle | 0.88 | 0.91 | 0.90 | 100 |
| caterpillar | 0.74 | 0.66 | 0.70 | 100 |
| cattle | 0.82 | 0.76 | 0.79 | 100 |
| chair | 0.90 | 0.87 | 0.88 | 100 |

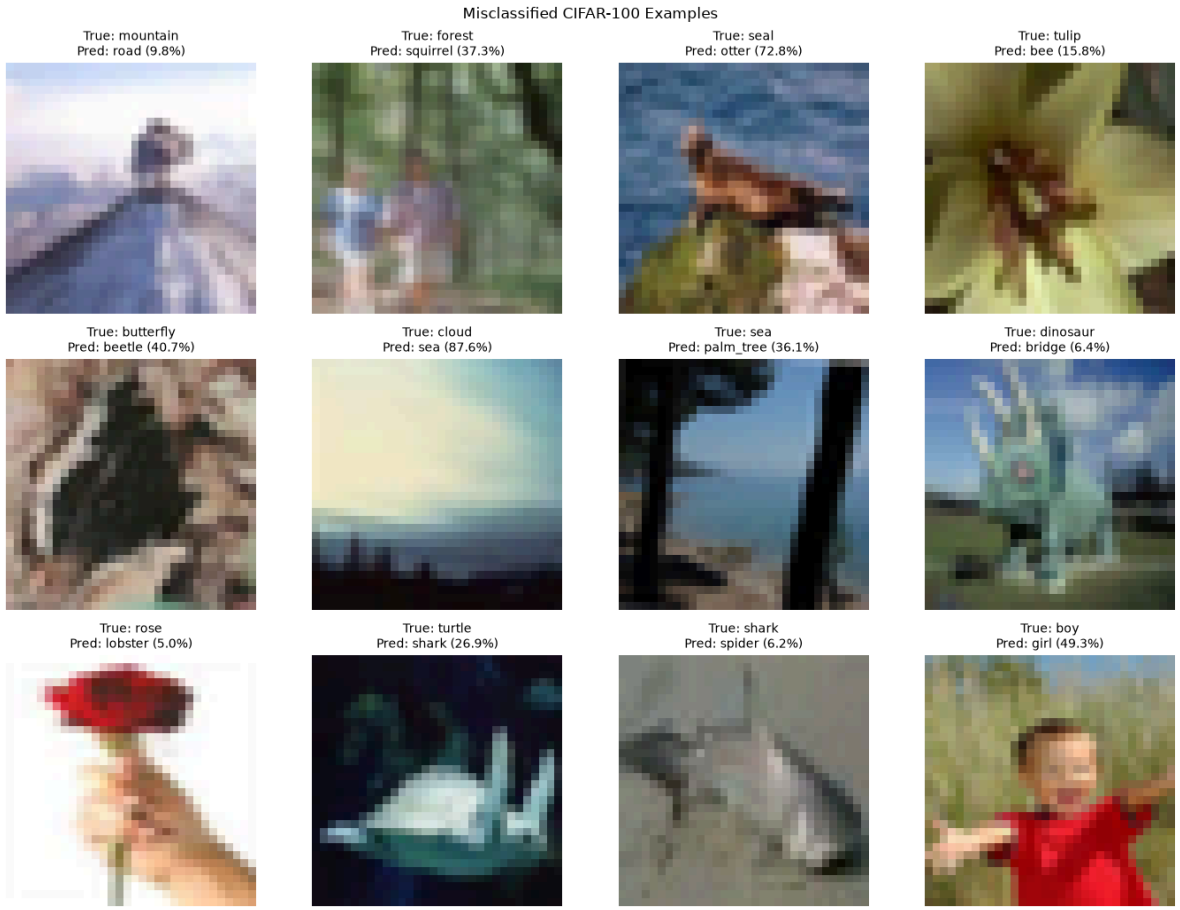
| | | | | |
|---|---|---|---|---|
| chimpanzee | 0.85 | 0.94 | 0.90 | 100 |
| clock | 0.82 | 0.81 | 0.81 | 100 |
| cloud | 0.88 | 0.84 | 0.86 | 100 |
| cockroach | 0.87 | 0.82 | 0.85 | 100 |
| couch | 0.75 | 0.72 | 0.73 | 100 |
| crab | 0.74 | 0.81 | 0.77 | 100 |
| crocodile | 0.67 | 0.68 | 0.68 | 100 |
| cup | 0.88 | 0.84 | 0.86 | 100 |
| dinosaur | 0.81 | 0.75 | 0.78 | 100 |
| dolphin | 0.74 | 0.76 | 0.75 | 100 |
| elephant | 0.78 | 0.73 | 0.75 | 100 |
| flatfish | 0.71 | 0.70 | 0.71 | 100 |
| forest | 0.71 | 0.67 | 0.69 | 100 |
| fox | 0.77 | 0.86 | 0.82 | 100 |
| girl | 0.59 | 0.55 | 0.57 | 100 |
| hamster | 0.88 | 0.84 | 0.86 | 100 |
| house | 0.81 | 0.77 | 0.79 | 100 |
| kangaroo | 0.78 | 0.75 | 0.77 | 100 |
| keyboard | 0.88 | 0.90 | 0.89 | 100 |
| lamp | 0.75 | 0.77 | 0.76 | 100 |
| lawn_mower | 0.92 | 0.89 | 0.90 | 100 |
| leopard | 0.79 | 0.82 | 0.80 | 100 |
| lion | 0.90 | 0.87 | 0.88 | 100 |
| lizard | 0.57 | 0.63 | 0.60 | 100 |
| lobster | 0.62 | 0.70 | 0.66 | 100 |

| | | | | |
|---|---|---|---|---|
| man | 0.58 | 0.64 | 0.61 | 100 |
| maple_tree | 0.63 | 0.69 | 0.66 | 100 |
| motorcycle | 0.91 | 0.97 | 0.94 | 100 |
| mountain | 0.87 | 0.86 | 0.86 | 100 |
| mouse | 0.60 | 0.67 | 0.63 | 100 |
| mushroom | 0.87 | 0.75 | 0.81 | 100 |
| oak_tree | 0.67 | 0.67 | 0.67 | 100 |
| orange | 0.88 | 0.96 | 0.92 | 100 |
| orchid | 0.82 | 0.90 | 0.86 | 100 |
| otter | 0.51 | 0.63 | 0.57 | 100 |
| palm_tree | 0.94 | 0.90 | 0.92 | 100 |
| pear | 0.73 | 0.80 | 0.76 | 100 |
| pickup_truck | 0.91 | 0.92 | 0.92 | 100 |
| pine_tree | 0.69 | 0.70 | 0.70 | 100 |
| plain | 0.89 | 0.93 | 0.91 | 100 |
| plate | 0.85 | 0.75 | 0.80 | 100 |
| poppy | 0.80 | 0.77 | 0.79 | 100 |
| porcupine | 0.80 | 0.78 | 0.79 | 100 |
| possum | 0.74 | 0.61 | 0.67 | 100 |
| rabbit | 0.67 | 0.66 | 0.66 | 100 |
| raccoon | 0.87 | 0.88 | 0.88 | 100 |
| ray | 0.73 | 0.63 | 0.68 | 100 |
| road | 0.90 | 0.95 | 0.93 | 100 |
| rocket | 0.87 | 0.84 | 0.85 | 100 |
| rose | 0.78 | 0.77 | 0.77 | 100 |

| | | | | |
|---|---|---|---|---|
| sea | 0.78 | 0.86 | 0.82 | 100 |
| seal | 0.56 | 0.54 | 0.55 | 100 |
| shark | 0.58 | 0.62 | 0.60 | 100 |
| shrew | 0.59 | 0.63 | 0.61 | 100 |
| skunk | 0.94 | 0.92 | 0.93 | 100 |
| skyscraper | 0.93 | 0.91 | 0.92 | 100 |
| snail | 0.80 | 0.72 | 0.76 | 100 |
| snake | 0.74 | 0.67 | 0.70 | 100 |
| spider | 0.77 | 0.80 | 0.78 | 100 |
| squirrel | 0.66 | 0.69 | 0.68 | 100 |
| streetcar | 0.78 | 0.83 | 0.81 | 100 |
| sunflower | 0.96 | 0.91 | 0.93 | 100 |
| sweet_pepper | 0.73 | 0.72 | 0.73 | 100 |
| table | 0.80 | 0.77 | 0.79 | 100 |
| tank | 0.89 | 0.90 | 0.90 | 100 |
| telephone | 0.82 | 0.77 | 0.79 | 100 |
| television | 0.78 | 0.87 | 0.82 | 100 |
| tiger | 0.91 | 0.84 | 0.88 | 100 |
| tractor | 0.85 | 0.92 | 0.88 | 100 |
| train | 0.85 | 0.91 | 0.88 | 100 |
| trout | 0.80 | 0.84 | 0.82 | 100 |
| tulip | 0.74 | 0.72 | 0.73 | 100 |
| turtle | 0.76 | 0.64 | 0.70 | 100 |
| wardrobe | 0.90 | 0.95 | 0.93 | 100 |
| whale | 0.80 | 0.76 | 0.78 | 100 |

| | | | | |
|---|---|---|---|---|
| willow_tree | 0.69 | 0.66 | 0.68 | 100 |
| wolf | 0.80 | 0.86 | 0.83 | 100 |
| woman | 0.58 | 0.59 | 0.59 | 100 |
| worm | 0.76 | 0.74 | 0.75 | 100 |

**Display Misclassified Images based on 12 Classes:**




#### 2.4.3.1.3 ResNet Pretrained - Full Finetune

Fine-tuning in machine learning is the process of adapting a pre-trained model for specific tasks or use cases. It has become a fundamental deep learning technique, particularly in the training process of foundation models used for generative AI. Fine-tuning could be considered a subset of the broader technique of transfer learning: the practice of leveraging knowledge an existing model has already learned as the starting point for learning new tasks.

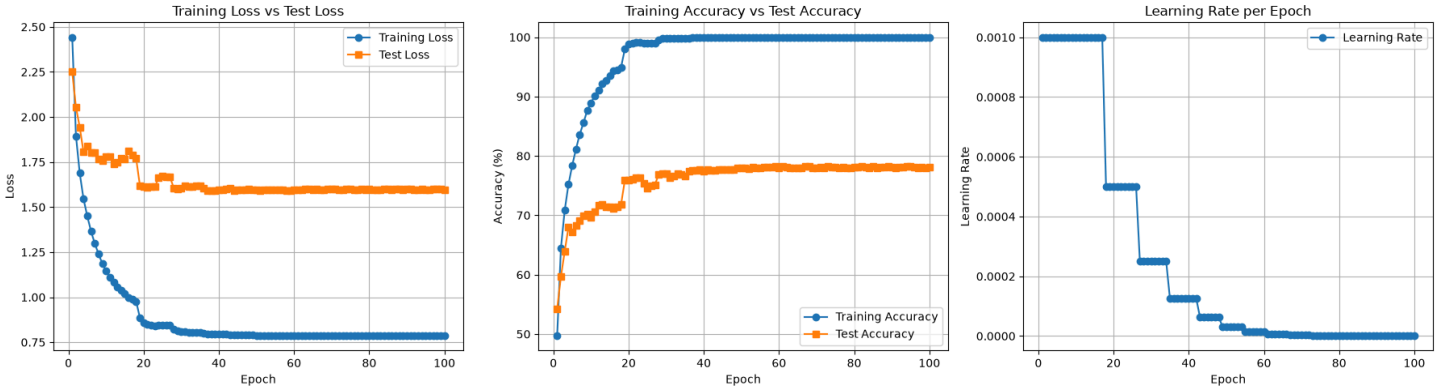
| **Freeze layers** | None |
|---|---|

| Unfreeze layers | conv1, bn1, layer1, layer2, layer3, layer4 |
|---|---|

*The table of Pretrained - Fine-tune Strategy*

- **Hyperparameters Configuration:**

| Epoch | Learning Rate | Batch Size | Train Loss | Test Loss | Train Accuracy | Test Accuracy | Cross Entropy Function | Trainable Parameters |
|---|---|---|---|---|---|---|---|---|
| 100 | 0.001 | 128 | 0.7863 | 1.5958 | 99.97% | 78.13% | AdamW(weight_decay=1e-3) | 11220132 |




*The training progress of ResNet-18 Pretrained Fine-tune*

The table accuracy of **100 classes** based on the **Test Dataset**

| Classes | Accuracy |
|---|---|
| apple | 93.0% |
| aquarium_fish | 89.0% |
| baby | 62.0% |
| bear | 65.0% |
| beaver | 69.0% |
| bed | 83.0% |
| bee | 83.0% |
| beetle | 81.0% |
| bicycle | 93.0% |
| bottle | 92.0% |

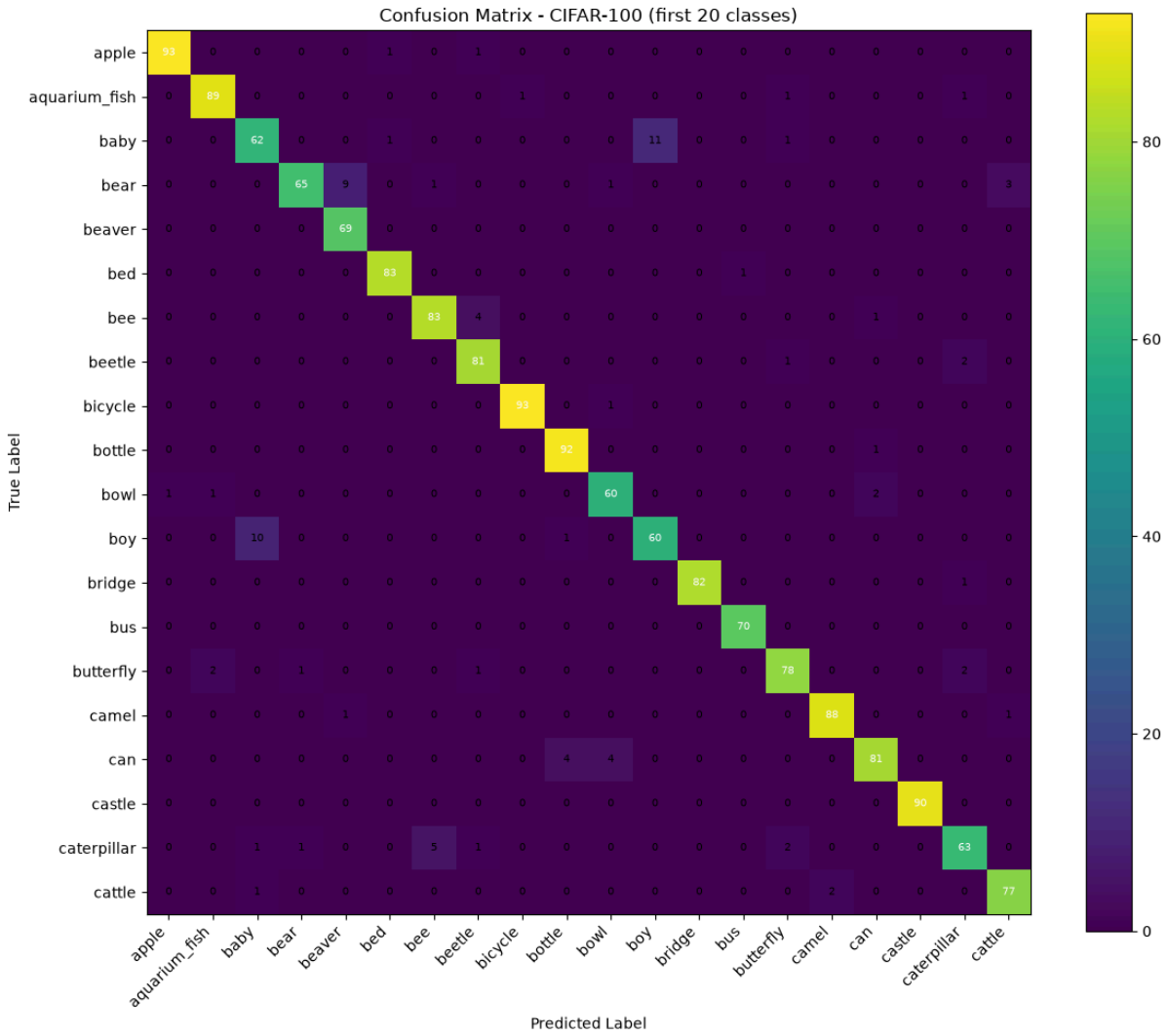
| | |
|---|---|
| bowl | 60.0% |
| boy | 60.0% |
| bridge | 82.0% |
| bus | 70.0% |
| butterfly | 78.0% |
| camel | 88.0% |
| can | 81.0% |
| castle | 90.0% |
| caterpillar | 63.0% |
| cattle | 77.0% |
| chair | 89.0% |
| chimpanzee | 92.0% |
| clock | 78.0% |
| cloud | 83.0% |
| cockroach | 85.0% |
| couch | 70.0% |
| crab | 79.0% |
| crocodile | 70.0% |
| cup | 82.0% |
| dinosaur | 77.0% |
| dolphin | 77.0% |
| elephant | 75.0% |
| flatfish | 69.0% |
| forest | 72.0% |
| fox | 79.0% |

| | |
|---|---|
| girl | 57.0% |
| hamster | 85.0% |
| house | 80.0% |
| kangaroo | 73.0% |
| keyboard | 90.0% |
| lamp | 76.0% |
| lawn_mower | 90.0% |
| leopard | 85.0% |
| lion | 83.0% |
| lizard | 57.0% |
| lobster | 75.0% |
| man | 55.0% |
| maple_tree | 74.0% |
| motorcycle | 96.0% |
| mountain | 89.0% |
| mouse | 73.0% |
| mushroom | 78.0% |
| oak_tree | 69.0% |
| orange | 93.0% |
| orchid | 85.0% |
| otter | 60.0% |
| palm_tree | 91.0% |
| pear | 85.0% |
| pickup_truck | 92.0% |
| pine_tree | 69.0% |

| | |
|---|---|
| plain | 90.0% |
| plate | 71.0% |
| poppy | 79.0% |
| porcupine | 77.0% |
| possum | 61.0% |
| rabbit | 66.0% |
| raccoon | 88.0% |
| ray | 68.0% |
| road | 95.0% |
| rocket | 85.0% |
| rose | 76.0% |
| sea | 89.0% |
| seal | 57.0% |
| shark | 62.0% |
| shrew | 57.0% |
| skunk | 94.0% |
| skyscraper | 95.0% |
| snail | 72.0% |
| snake | 64.0% |
| spider | 78.0% |
| squirrel | 70.0% |
| streetcar | 84.0% |
| sunflower | 92.0% |
| sweet_pepper | 75.0% |
| table | 73.0% |

| | |
|---|---|
| tank | 90.0% |
| telephone | 79.0% |
| television | 86.0% |
| tiger | 88.0% |
| tractor | 92.0% |
| train | 86.0% |
| trout | 85.0% |
| tulip | 69.0% |
| turtle | 67.0% |
| wardrobe | 96.0% |
| whale | 77.0% |
| willow_tree | 68.0% |
| wolf | 78.0% |
| woman | 64.0% |
| worm | 74.0% |

**Confusion Matrix of 20 Classes**




**Classification Metric**

| Accuracy | Precision | Recall | F1-score |
|---|---|---|---|
| 78.13% | 0.7846 | 0.7813 | 0.7818 |

| Classes | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| apple | 0.91 | 0.93 | 0.92 | 100 |
| aquarium_fish | 0.89 | 0.89 | 0.89 | 100 |
| baby | 0.68 | 0.62 | 0.65 | 100 |
| bear | 0.68 | 0.65 | 0.66 | 100 |
| beaver | 0.60 | 0.69 | 0.64 | 100 |
| bed | 0.80 | 0.83 | 0.81 | 100 |

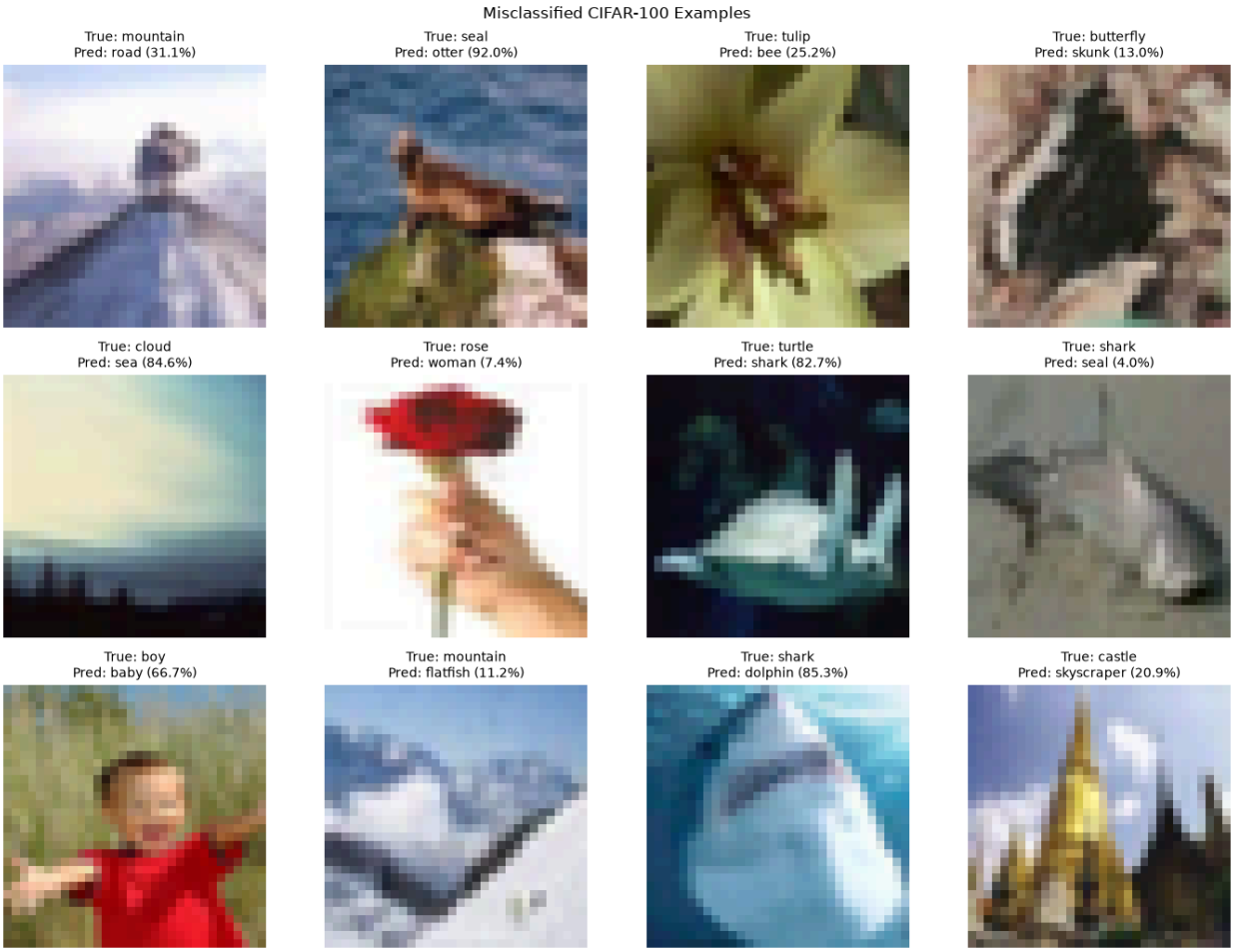
| | | | | |
|---|---|---|---|---|
| bee | 0.84 | 0.83 | 0.83 | 100 |
| beetle | 0.79 | 0.81 | 0.80 | 100 |
| bicycle | 0.95 | 0.93 | 0.94 | 100 |
| bottle | 0.88 | 0.92 | 0.90 | 100 |
| bowl | 0.60 | 0.60 | 0.60 | 100 |
| boy | 0.54 | 0.60 | 0.57 | 100 |
| bridge | 0.88 | 0.82 | 0.85 | 100 |
| bus | 0.86 | 0.70 | 0.77 | 100 |
| butterfly | 0.80 | 0.78 | 0.79 | 100 |
| camel | 0.82 | 0.88 | 0.85 | 100 |
| can | 0.83 | 0.81 | 0.82 | 100 |
| castle | 0.93 | 0.90 | 0.91 | 100 |
| caterpillar | 0.72 | 0.63 | 0.67 | 100 |
| cattle | 0.81 | 0.77 | 0.79 | 100 |
| chair | 0.90 | 0.89 | 0.89 | 100 |
| chimpanzee | 0.87 | 0.92 | 0.89 | 100 |
| clock | 0.74 | 0.78 | 0.76 | 100 |
| cloud | 0.86 | 0.83 | 0.85 | 100 |
| cockroach | 0.92 | 0.85 | 0.89 | 100 |
| couch | 0.69 | 0.70 | 0.70 | 100 |
| crab | 0.71 | 0.79 | 0.75 | 100 |
| crocodile | 0.64 | 0.70 | 0.67 | 100 |
| cup | 0.85 | 0.82 | 0.83 | 100 |
| dinosaur | 0.84 | 0.77 | 0.80 | 100 |
| dolphin | 0.69 | 0.77 | 0.73 | 100 |

| | | | | |
|---|---|---|---|---|
| elephant | 0.84 | 0.75 | 0.79 | 100 |
| flatfish | 0.74 | 0.69 | 0.72 | 100 |
| forest | 0.70 | 0.72 | 0.71 | 100 |
| fox | 0.81 | 0.79 | 0.80 | 100 |
| girl | 0.57 | 0.57 | 0.57 | 100 |
| hamster | 0.86 | 0.85 | 0.85 | 100 |
| house | 0.81 | 0.80 | 0.80 | 100 |
| kangaroo | 0.74 | 0.73 | 0.73 | 100 |
| keyboard | 0.92 | 0.90 | 0.91 | 100 |
| lamp | 0.79 | 0.76 | 0.78 | 100 |
| lawn_mower | 0.94 | 0.90 | 0.92 | 100 |
| leopard | 0.81 | 0.85 | 0.83 | 100 |
| lion | 0.87 | 0.83 | 0.85 | 100 |
| lizard | 0.48 | 0.57 | 0.52 | 100 |
| lobster | 0.68 | 0.75 | 0.71 | 100 |
| man | 0.58 | 0.55 | 0.56 | 100 |
| maple_tree | 0.65 | 0.74 | 0.69 | 100 |
| motorcycle | 0.91 | 0.96 | 0.93 | 100 |
| mountain | 0.89 | 0.89 | 0.89 | 100 |
| mouse | 0.68 | 0.73 | 0.70 | 100 |
| mushroom | 0.81 | 0.78 | 0.80 | 100 |
| oak_tree | 0.66 | 0.69 | 0.68 | 100 |
| orange | 0.89 | 0.93 | 0.91 | 100 |
| orchid | 0.83 | 0.85 | 0.84 | 100 |
| otter | 0.45 | 0.60 | 0.52 | 100 |

| | | | | |
|---|---|---|---|---|
| palm_tree | 0.94 | 0.91 | 0.92 | 100 |
| pear | 0.79 | 0.85 | 0.82 | 100 |
| pickup_truck | 0.89 | 0.92 | 0.91 | 100 |
| pine_tree | 0.78 | 0.69 | 0.73 | 100 |
| plain | 0.86 | 0.90 | 0.88 | 100 |
| plate | 0.81 | 0.71 | 0.76 | 100 |
| poppy | 0.81 | 0.79 | 0.80 | 100 |
| porcupine | 0.83 | 0.77 | 0.80 | 100 |
| possum | 0.78 | 0.61 | 0.69 | 100 |
| rabbit | 0.79 | 0.66 | 0.72 | 100 |
| raccoon | 0.80 | 0.88 | 0.84 | 100 |
| ray | 0.77 | 0.68 | 0.72 | 100 |
| road | 0.90 | 0.95 | 0.92 | 100 |
| rocket | 0.86 | 0.85 | 0.85 | 100 |
| rose | 0.82 | 0.76 | 0.79 | 100 |
| sea | 0.83 | 0.89 | 0.86 | 100 |
| seal | 0.52 | 0.57 | 0.54 | 100 |
| shark | 0.58 | 0.62 | 0.60 | 100 |
| shrew | 0.64 | 0.57 | 0.60 | 100 |
| skunk | 0.91 | 0.94 | 0.93 | 100 |
| skyscraper | 0.90 | 0.95 | 0.92 | 100 |
| snail | 0.77 | 0.72 | 0.75 | 100 |
| snake | 0.67 | 0.64 | 0.65 | 100 |
| spider | 0.80 | 0.78 | 0.79 | 100 |
| squirrel | 0.74 | 0.70 | 0.72 | 100 |

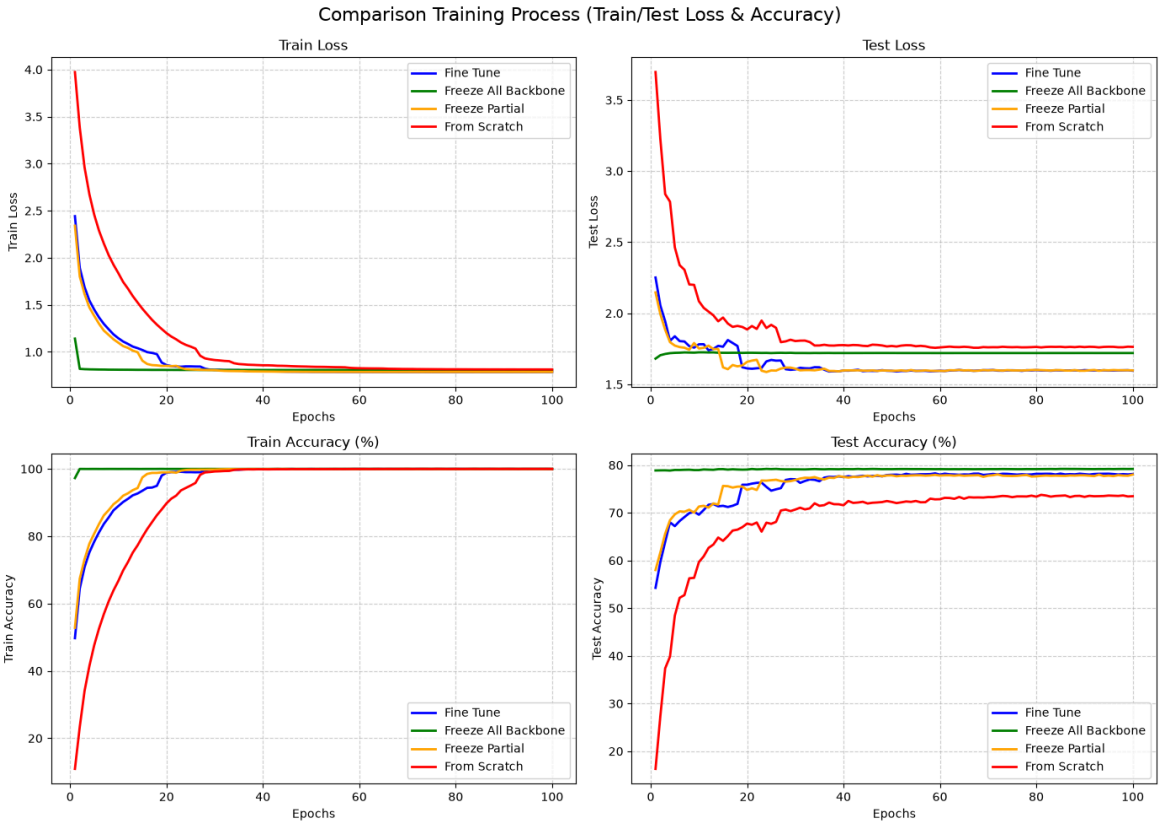
| | | | | |
|---|---|---|---|---|
| streetcar | 0.76 | 0.84 | 0.80 | 100 |
| sunflower | 0.94 | 0.92 | 0.93 | 100 |
| sweet_pepper | 0.82 | 0.75 | 0.78 | 100 |
| table | 0.79 | 0.73 | 0.76 | 100 |
| tank | 0.87 | 0.90 | 0.89 | 100 |
| telephone | 0.84 | 0.79 | 0.81 | 100 |
| television | 0.77 | 0.86 | 0.81 | 100 |
| tiger | 0.86 | 0.88 | 0.87 | 100 |
| tractor | 0.90 | 0.92 | 0.91 | 100 |
| train | 0.77 | 0.86 | 0.82 | 100 |
| trout | 0.87 | 0.85 | 0.86 | 100 |
| tulip | 0.68 | 0.69 | 0.68 | 100 |
| turtle | 0.79 | 0.67 | 0.72 | 100 |
| wardrobe | 0.91 | 0.96 | 0.94 | 100 |
| whale | 0.79 | 0.77 | 0.78 | 100 |
| willow_tree | 0.69 | 0.68 | 0.69 | 100 |
| wolf | 0.85 | 0.78 | 0.81 | 100 |
| woman | 0.60 | 0.64 | 0.62 | 100 |
| worm | 0.76 | 0.74 | 0.75 | 100 |

**Display Misclassified Images based on 12 Classes:**




## 2.4.4 Comparison And Evaluation

- **Learning Progress Data**



| | From-Scratch | Pretrained Model | | |
|---|---|---|---|---|
| | | Freeze All Backbone | Freeze Partial Backbone | Fine-Tune |
| **Epochs** | 100 | | | |
| **Learning Rate** | 0.001 | | | |
| **Cross Entropy Function** | AdamW(weight_decay=1e-3) | | | |
| **Batch Size** | 128 | | | |
| **Train Loss** | 0.8137 | 0.8077 | 0.7867 | 0.7863 |
| **Test Loss** | 1.7636 | 1.7202 | 1.5979 | 1.5958 |
| **Train Accuracy** | 99.97% | 99.98% | 99.98% | 99.97% |
| **Test Accuracy** | 73.47% | 79.19% | 78.01% | 78.13% |

- **Evaluations**

| | From-Scratch | Pretrained Model | | |
|---|---|---|---|---|
| | | Freeze All Backbone | Freeze Partial Backbone | Fine-Tune |
| **Behavior** | Starts with random weight initialization (initial train accuracy ~10.9%, matching random chance for a 10-class task). Train loss starts high (~3.97) and requires ~32 epochs to reach near 100% training accuracy. | Only the classification head is trained while the feature extractor backbone remains completely locked. Reaches 100% training accuracy almost immediately (by Epoch 2) and maintains constant performance throughout training. | Keeps early/lower-level feature layers locked while updating higher-level feature blocks and the classifier head. Initial accuracy starts around ~52.8% and steadily reaches 100% around Epoch 25. | Updates all model parameters end-to-end, initialized from pretrained weights. Starts around ~49.8% initial train accuracy and reaches 100% train accuracy around Epoch 25. |
| **Performance** | Reaches the lowest final test accuracy (~73.5%) and highest test loss (~1.76). | Reaches the highest final test accuracy (~79.2%), with test accuracy starting high (~78.8%) right at | Reaches ~78.0% test accuracy with a final test loss of ~1.60. | Reaches ~78.1% test accuracy with a final test loss of ~1.60. |

| | From-Scratch | Pretrained Model | | |
|---|---|---|---|---|
| | | Epoch 1. | | |
| **Summary** | Demonstrates the limits of training complex models without prior representations on smaller or domain-constrained datasets. Suffer from high generalization error due to learning feature extractors entirely from scratch. | Most computationally efficient and fastest to converge. Prevents catastrophic forgetting of pretrained visual primitives, making it highly effective when the target dataset closely matches the pretraining domain. | Achieves lower test loss (~1.60 vs ~1.73) compared to freezing the entire backbone, indicating better-calibrated confidence scores. Provides a strong compromise when higher-level features need adaptation to target-specific visual patterns. | Shows nearly identical convergence and test metrics to partial backbone freezing. While flexible, full fine-tuning requires higher compute and careful learning rate scheduling to prevent destabilizing pretrained representations. |

# 3. References

Asad, Muhammad. “Introduction to Softmax Classifier in PyTorch - MachineLearningMastery.com.” *Machine Learning Mastery*, https://machinelearningmastery.com/introduction-to-softmax-classifier-in-pytorch/. Accessed 15 October 2026.